https://hjfy.top/arxiv/2609.14858v1

Google

梦境-RSI：通过演化世界实现递归自我提升

Tong Zheng[1,2], Xidong Wu[1], Zheng Zhang[1], Zhankui He[3], Chaoyi Zhang[1], Benjamin Coleman[3], Ruoqiao Wei[1], Di Bai[3], Haolin Liu[4], Rui Liu[2], Xue Wang[1], Yue Zhuan[1], Wang-Cheng Kang[3], Renkai Xiang[1], Heng Huang[2], Xinwu Cheng[1] and Yunsong Guo[1]

[1]Google, [2]University of Maryland, College Park, [3]Google Deepmind, [4]University of Virginia

递归自我改进正日益成为自主智能体的关键，其进展依赖于在复杂领域中发现高价值解的能力。这一过程的核心驱动力是有效的探索，然而管理和优化探索策略仍是主要瓶颈。当前系统面临一个根本性困境：固定策略无法随搜索空间的扩大而自适应调整，而在线策略最优化则需在延迟且昂贵的反馈下，于长时程滚动中导航庞大的元搜索空间。我们提出了 Dream-RSI，一种可扩展且递归自我改进的探索框架。一个轻量级编排层使探索变得显式且可编程，同时保持底层编码智能体不变。我们的关键洞见是：累积的发现历史可作为对已实现搜索空间的回放模拟器。通过在由历史发现树构建的回放模拟器中进行“梦境”推演，Dream-RSI 能够获得即时、低成本的异策略反馈，以评估和优化探索策略，而无需重复执行昂贵的在线评估。优化后的策略随后被重新部署至线上，驱动进一步发现，从而持续扩展模拟器池，形成自我改进的闭环。在算法工程、数学优化及 GPU 内核工程等多个情景中，Dream-RSI 在多个设置下均实现了具有竞争力或更优的发现质量，同时显著降低了发现成本。

github.com/zhengkid/Dream-RSI | dream-rsi.com

1. 引言

递归自我改进（RSI）已成为自主人工智能系统的一个雄心勃勃的目标 (Liu et al., 2026c)。支撑 RSI 的常见机制是一种迭代发现环，其中智能体生成候选解决方案，评估结果，融入反馈，并优化未来的迭代。这种发现环已推动科学和算法领域取得显著进展，包括算法设计 (Novikov et al., 2025; Romera-Paredes et al., 2024)、开放式的数学优化 (Anthropic, 2026; Georgiev et al., 2025)、系统设计 (Cao et al., 2026; Jaber and Jaber, 2026) 以及智能体自我改进 (Lee et al., 2026; Zhang et al., 2026b,c; Zheng et al., 2026a)，这些发现正越来越多地用于更强大人工智能系统的开发。随着智能体能力的提升和自我改进目标变得更具挑战性，发现过程越来越需要在广阔搜索空间中进行长时程探索，通常涉及数千次提案-评估循环 (OpenAI, 2026; Ye et al., 2026)。在此规模下，协调探索的能力变得至关重要 (Zheng et al., 2026b)。低效的探索会浪费大量计算资源和时间，严重限制 RSI 的效率与可扩展性。

现有方法主要依赖于手动设计的探索策略，这些策略在发现过程中基本保持不变 (Du et al., 2026; Jiang et al., 2026; Novikov et al., 2025; Yan et al., 2026b; Ye et al., 2026)。固定策略无法从积累的发现经验中改进，可能反复将计算资源分配给无效的搜索方向。因此，近期工作试图在发现过程中在线优化探索策略 (Liu et al., 2026a)，但这样做面临两个根本性瓶颈。首先，元层面的反馈具有延迟且成本高昂：与评估单个候选方案不同，评估一个探索策略需要观察其在多个提案-评估循环中如何影响后续的发现过程。其次，元策略空间极为庞大：新提出的策略可能表现不佳，因此需要尝试许多替代方案。综上所述，这些挑战使得元层面的改进尤为昂贵：每个策略可能都需要长时间的在线滚动执行才能获得有用反馈，从而难以高效地在探索层闭合自我改进环。

Corresponding author(s): xidongwu@google.com, zzhangx@google.com
© 2026 Google. All rights reserved

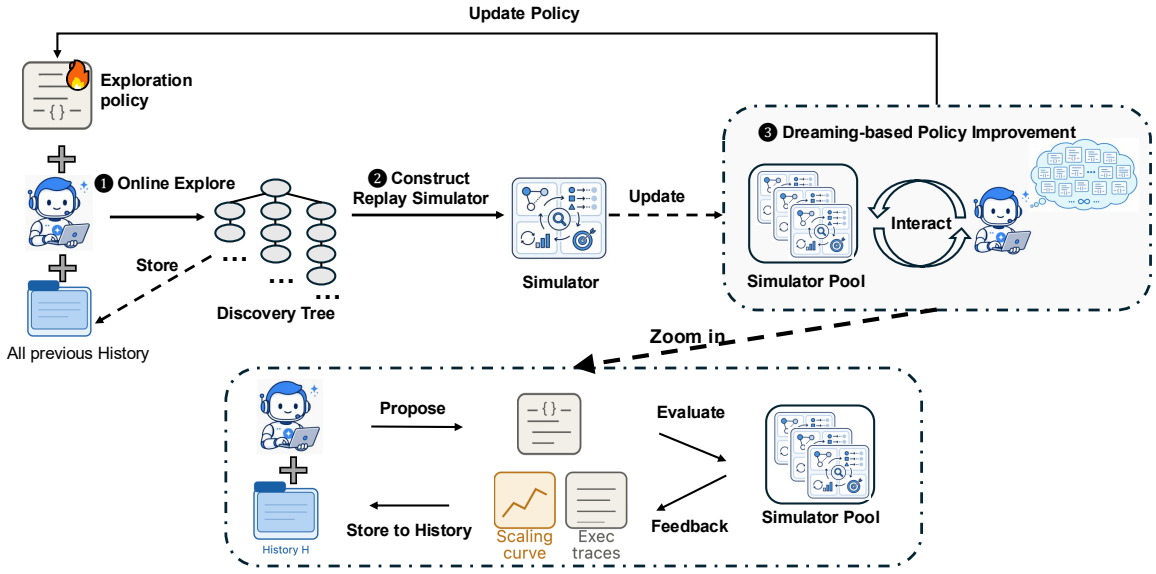


Figure 1 | **Dream-RSI 系统概览**。该系统通过三个核心阶段实现递归的自我改进循环：**① 在线探索**，当前探索策略引导编码智能体扩展发现树并记录历史迹；**② 构建回放模拟器**，将生成的发现树转化为可重复使用的模拟器池；以及 **③ 基于“梦境”的策略改进**，智能体在脑海中“构想”出大量备选策略。随后，这些候选策略被输入回放模拟器以进行执行模拟并获取快速反馈，持续优化其策略（详见缩放框内详细说明）。更新后的策略随即重新部署，进入下一轮在线探索。

为解决这些瓶颈，我们的核心直觉很简单：一个快速且低成本的发现模拟器将允许在昂贵的在线部署前评估多种探索策略。**令人惊讶的是，已完成的发现历史本身已具备这样的模拟器功能**。尽管先前的工作仅将过去的发现历史视为静态文本上下文 (Hu et al., 2025; Ouyang et al., 2026b) 或用于权重微调的训练数据 (Wang et al., 2025; Yuksekgonul et al., 2026)，但一个完整的发现过程本质上记录了过去探索决策及其实际代码执行结果的结构化树状信息。类比于基于模型的强化学习和世界模型 (Ha and Schmidhuber, 2018; Hafner et al., 2023) (§2)，一旦将其组织成发现树，该历史便可作为**回放模拟器** [1]. 如图 2 所示，一种替代的探索策略可以导航此预先记录的树，以不同顺序遍历记录分支的不同子集，采用不同的并行分组方式和停止决策。由于所有执行结果均已保存在树中，评估新策略仅需读取历史记录，无需重新运行潜在的发现智能体或评估器。这将元策略改进从昂贵的在线试验-错误过程转变为快速的、基于仿真的“梦境”过程。

基于这一洞察，我们提出了 Dream-RSI，一种在智能体驱动发现中可扩展且递归自提升的元探索框架。我们首先通过一个轻量级编排层，将探索显式化并实现编程控制，该层负责分支、并行探索以及停止决策，同时保持底层编码智能体不变。与固定策略不同，Dream-RSI 在三个核心阶段建立了一个闭环自提升机制（图 1）：**(1) 在线探索**，当前策略指导真实世界中的发现，并记录历史执行迹；**(2) 模拟器构建**，将记录的发现树转换为可复用的回放模拟器池；以及 **(3) 基于“梦想”的策略改进**，通过在模拟器上低成本的“梦想”对候选策略进行评估。更新后的策略随后被重新部署到线上，生成新的发现经验并扩展模拟器池，从而在元探索层完成一个 RSI 环路。

[1]我们交替使用“回放模拟器”和“世界”这两个术语。

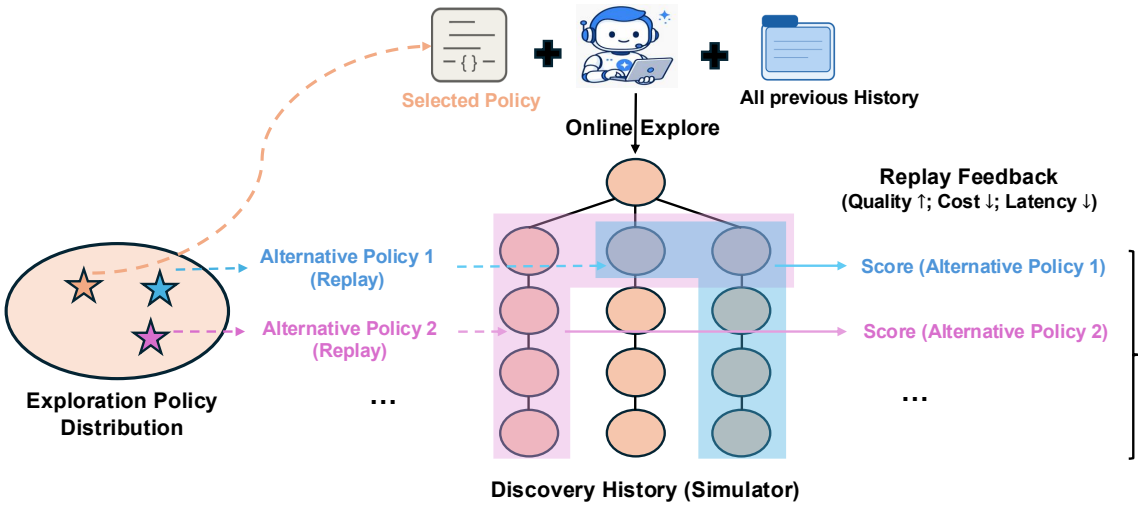


Figure 2 | **发现历史作为回放模拟器**。部署的策略首先在线探索，生成包含历史执行迹的结构化发现树（每个结点表示一次尝试及其完整的观测）。随后，可在该模拟器中测试数千种候选策略——评估搜索分支、探索顺序、并发级别和停止规则等不同选择。**由于所有结点结果均已预先存储，单次昂贵的在线运行即可实现数千次快速、零执行成本的异策略评估。这使得通过历史回放进行策略改进成为可能：智能体可以在重新在线部署改进后的策略前，‘梦想’多种不同的探索策略**。

在实验上，我们在涵盖三个不同领域的 8 个科学发现任务中评估了DREAM-RSI：算法工程、数学优化和 GPU 内核工程。在算法工程（Lasso 路径求解器）中，DREAM-RSI的表现优于标准库如 sklearn 以及强基准方法，同时将智能体调用次数相比SIMPLETES减少了高达 162×%，相比固定探索基准减少了 1.7×%。在数学优化（和差问题、自相关、圆排列）中，它在 1k 代内即达到或超越强基准性能，相较于SIMPLETES实现了超过 50× 的预算节省。在 GPU 内核工程（KernelBench）中，它要么在使用 1.79×–2.43× 代更少的情况下达到目标执行速度，要么在相同的预算约束下将内核性能提升高达 2.09×。

总结而言，我们的主要贡献如下：1) **历史作为回放模拟器：**我们将已完成的发现历史概念化为回放模拟器。这使得延迟的探索反馈可被重用，以实现高效的元探索策略评估；2) **元层 RSI 环（DREAM-RSI）：**我们引入了DREAM-RSI，建立了一个递归自我改进环，通过在线探索持续收集发现历史，从历史中构建回放模拟器，利用“梦想”来优化元探索策略，并在线重新部署升级后的策略；3) **实验验证：**我们进行了实验，证明在多种情景下，DREAM-RSI能够提升发现的有效性和效率。

2. 动机：发现历史作为回放模拟器

考虑一个智能体在陌生环境中向目标位置导航。在首次遍历过程中，该智能体可能会选择低效的路径，遭遇死胡同，回溯，并逐步构建周围空间的地图。然而，一旦记录下来，这种经验即可重复使用：生成的地图支持规划，无需智能体物理上重新访问每个位置。新的导航策略可以基于累积的地图进行推理，避开已知的死胡同，重新考虑早期决策，并在行动前比较替代路径 (Gupta et al., 2017)。

这一思想与基于模型的强化学习 (M. Moerland et al., 2023; Sutton, 1990) 相似。模型捕捉了环

境在智能体动作影响下的演化过程，使得策略可以通过模拟经验进行训练或评估，而无需反复与真实环境交互 (Ha and Schmidhuber, 2018)。Dreamer 系列 (Hafner et al., 2019, 2020, 2023, 2025) 特别清晰地展示了这一原理：智能体从收集的经验中学习一个紧凑的动力学模型，并通过在该模型内想象轨迹来改进其策略。

长时程发现同样具有类似的结构。探索策略决定应追求哪些方向，应优化哪些候选方案，应并行探索哪些分支，以及何时终止。在线执行该策略会生成一个包含已探索分支、决策点、计算成本和实际结果的结构化发现历史。如图 2 所示，该历史随后可被视作一个*经验回放模拟器*：对已观测发现空间部分的具身化模型。

在此回放模拟器中，不同的探索策略会引导出通过记录的发现树的不同轨迹。一种策略可能选择不同的分支子集，以不同的顺序优先处理这些分支，同时发出不同的并行请求，或在更早的节点处停止。评估此类轨迹只需揭示所选分支上已存储的结果，而无需重新运行潜在的编码智能体和评估器。因此，一次昂贵的在线发现运行即可支持对多种探索策略的众多低成本评估。

3. Dream-RSI：通过不断演化的世界实现递归自我提升

如图 1 所示，Dream-RSI 在在线探索与离线“梦境”之间交替进行，以改进一个可执行的*探索策略*，该策略负责分配发现计算。在在线阶段，该策略指导一个固定的*发现智能体*，同时一个固定的*评估器*对生成的候选方案进行评分，并提供诊断反馈。由此产生的*发现树*作为一个*回放世界*，其中可以使用记录的结果来评估其他策略。一个固定的基于大语言模型的*策略开发智能体*利用这些反馈来修改*探索策略*的代码，最佳评估版本将被部署用于下一次在线滚动。只有探索策略的代码发生变化；潜在的模型、评估器和执行接口保持不变。

发现树与共享决策界面。 发现树以 r 为根结点，代表初始的工作区状态。每个非根结点 v 恰好有一个*主要父结点*，该父结点要么是根结点，要么是之前创建的某个结点。此父结点标识了 v 中尝试的起始位置：发现智能体将恢复父结点保存的工作区，并使用其累积的观测结果作为上下文，生成新的尝试。结点 v 保留这一继承的历史记录，并记录新生成——评估尝试的结果，包括生成的文件系统快照、生成的成果物、评估诊断信息以及得分 s_v 。得分遵循固定的任务评分协议，数值越大表示质量越高。

在在线执行和离线回放中，探索策略观察一棵树 $\mathcal{T}$，该树初始时仅包含根结点，并选择从中继续探索的结点。可选结点构成集合 $A(\mathcal{T}) = \{r\} \cup \{v \in \mathcal{T} : v \text{ is a leaf}\}$，其中叶结点由当前观测到的树确定。令 $W \geq 1$ 为并行工作进程的数量，每个工作进程一次可执行一个生成-评估请求（例如并发 API 调用）。探索策略的动作是一个批量 $C \in A(\mathcal{T}; W)$，其中 $A(\mathcal{T}; W) = \{C \subseteq A(\mathcal{T}) : |C| \leq W\}$ 为可行批量集。每个选定的结点指定一次尝试的起始点，因此批量同时决定了探索继续的位置以及并行调度的尝试次数。在线和离线阶段均使用相同的决策接口，但在选择批量后所发生的转移有所不同。

在线部署。 令 $t = 1, 2, \ldots$ 表示外层迭代，从初始策略 π_1 和空的历史记录 $\mathcal{H}_0 = ()$ 开始。在迭代 t 时，策略 π_t 指导一次新的在线回溯，可访问已完成的发现历史 $\mathcal{H}_{t-1}$ 。该历史记录为探索提供上下

文，但与正在构建的新树保持分离。在整个回溯过程中，策略代码保持不变。

令 $\mathcal{T}_t^k$ 表示在 k 完成决策轮次后的新发现树，包含 $\mathcal{T}_t^0 = \{r\}$ 。回溯过程最多允许 K_1 轮。在第 $k \le K_1$ 轮，探索策略选择一个结点批量 $C_t^k \in A(\mathcal{T}_t^k; W)$ ，每个结点 $v \in C_t^k$ 被分配给一个工作节点。发现智能体利用 v 保存的工作空间和可用上下文生成一个新的候选结果，评估器则对结果进行评估。这些尝试并行执行，每个生成其选定父结点的一个新子结点。将已完成的子结点附加到当前树上，得到 $\mathcal{T}_t^{k+1}$ ，而所有先前记录的结点保持不变。该转移过程是随机的，因为发现智能体可能从相同起始工作空间生成不同结果。在下一回合中，新创建的子结点成为扩展分支的可选叶结点，而根结点仍保持可选以开启更多分支。当策略选择空批量或完成 K_1 轮决策时，回溯过程终止。回溯结束后，其最终树被记录为 $\mathcal{T}_t$ 并追加至历史记录中，形成 $\mathcal{H}_t = \mathcal{H}_{t-1} \cup \{\mathcal{T}_t\}$ 。随后，该方法进入使用此扩展的重放世界集合的离线阶段。

离线评估。 在外部迭代 t 的离线阶段，历史 $\mathcal{H}_t$ 保持固定，同时该方法构建并评估 $M \ge 1$ 策略版本 $\pi_t^0, \ldots, \pi_t^{M-1}$ ，从 $\pi_t^0 = \pi_t$ 开始。每个版本都在每一个历史树 $\mathcal{T}_i$ ，$i = 1, \ldots, t$ 上分别进行评估，然后根据所得反馈开发下一个版本。我们使用 m 来索引策略版本，i 来索引重放世界，k 来统计单个策略—世界评估中的决策回合数。在此阶段，外层索引 t 固定不变，并在重放轨迹和得分的记号中省略。

对于每个策略-树对 (m, i)，重放会重置策略的每轮状态，并从 $\mathcal{T}_i^{m,0} = \{r\}$ 开始。此处，$\mathcal{T}_i^{m,k} \subseteq \mathcal{T}_i$ 表示在 k 完成若干轮后揭示出的子树。完整的记录树 $\mathcal{T}_i$ 保持不变；只有策略所观察到的部分会发生变化。在每个决策点，π_t^m 使用已揭示的观测结果选择一个批量 $C_i^{m,k} \in A(\mathcal{T}_i^{m,k}; W)$。与在线执行不同，重放会确定性地返回所选节点的记录子节点，而不是生成新的候选节点。在 探索策略执行一个非空批量 $C_i^{m,k}$ 后，下一个被观测到的树是 $\mathcal{T}_i^{m,k+1} = \mathcal{T}_i^{m,k} \cup \bigcup_{v \in C_i^{m,k}} \mathrm{Child}(v; \mathcal{T}_i, \mathcal{T}_i^{m,k})$，其中 $\mathrm{Child}(v; \mathcal{T}_i, \mathcal{T}_i^{m,k})$ 表示在当前已观测树 $T_i^{m,k}$ 的基础上，树 $\mathcal{T}_i$ 上 v 未被观测的子节点集合。对于 $v \ne r$，$\mathrm{Child}(v; \mathcal{T}_i, \mathcal{T}_i^{m,k})$ 是 v 的唯一记录子节点（如果存在）。由于 v 是 $\mathcal{T}_i^{m,k}$ 的叶节点，该子节点仍未被揭示。对于 $v = r$，重放返回 r 在 $\mathcal{T}_i^{m,k}$ 之外最早创建的子节点，从而开启一条此前未被揭示的分支。在任一情况下，当没有剩余的记录延续时，$\mathrm{Child}(v; \mathcal{T}_i, \mathcal{T}_i^{m,k}) = \emptyset$。新揭示的节点会在策略做出下一步决策前暴露其存储的观测结果。

重放最多允许 K_2 个决策回合，其中每个非空批量计为一个回合，当策略选择 $C_i^{m,k} = \emptyset$ 、回合上限 $k = K_2$ 达到或 $\mathcal{T}_i^{m,k} = \mathcal{T}_i$ 时终止，表示所有记录的结点均已揭示。令 $k_i^{m,\star} \in \{0, \ldots, K_2\}$ 表示终止时已完成的回合数，从而得到最终子树 $\mathcal{T}_i^{m,k_i^{m,\star}} \subseteq \mathcal{T}_i$ 。

因此，回放评估每个已打开分支应推进多远、如何将尝试分组为并行批量，以及何时打开另一个分支或停止。这些决策可能因策略而异，但每个分支均按其记录的父-子顺序遍历，且不会生成超出 $\mathcal{T}_i$ 的结果。

重播目标。 重放目标平衡了发现质量、执行代价和并行性。令 $N_i^m = |\mathcal{T}_i^{m,k_i^{m,\star}}| - 1$ 为已揭示的非根结点数量。尽管重放本身并不执行新的发现尝试，N_i^m 却统计其轨迹所代表的生成–评估请求次数。对

于固定的系数 $\beta_1, \beta_2 \geq 0$ ，重放得分为

$$V_i^m = \underbrace{\max_{v \in \mathcal{T}_i^{m,k_i^{m,\star}}} s_v}_{\text{discovery quality}} - \underbrace{\beta_1 N_i^m}_{\text{execution cost}} + \underbrace{\beta_2 \frac{N_i^m}{\max\{1, k_i^{m,\star}\}}}_{\text{parallelism bonus}}. \tag{1}$$

第一项衡量的是在重放过程中达到的最佳解的质量。第二项惩罚了尝试生成的次数。对于非空重放，第三项奖励每决策轮次执行的平均尝试次数，这有利于那些批量处理有用延续而非顺序执行的策略。

策略改进与选择。 策略版本 π_t^m 的评估得分是其在固定历史记录中的平均回放得分，$V^m = \frac{1}{t}\sum_{i=1}^{t} V_i^m$。离线阶段从评估当前策略 $\pi_t^0 = \pi_t$ 开始。对于每个 $m = 0, \ldots, M-1$，策略开发智能体会检查 π_t^m 的回放轨迹和得分，以及早期修订的反馈，以识别成功的决策和反复出现的失败。然后，它会修改可执行的策略代码，生成 π_t^{m+1}，并在相同的 t 回放世界中对其进行评估。在各次修订之间，开发智能体可以获取回放反馈。

经过 M 次修订后，从所有 M 评估的版本中选择下一个在线策略为 $\pi_{t+1} = \pi_t^{m^\star}$ ，其中 $m^\star \in \arg\max_{m \in \{0,\ldots,M-1\}} V^m$ 。由于候选集包含当前策略，该选择满足 $V^{m^\star} \geq V^0$。因此，所选策略 π_{t+1} 在固定历史 $\mathcal{H}_t$ 上的平均回放得分上不劣于当前策略 π_t。然后将所选策略部署到线上以收集 $\mathcal{T}_{t+1}$，从而扩展下一次离线优化阶段可用的历史数据。

4. 实验

我们在三个科学发现领域对 Dream-RSI 进行评估：算法工程、内核最优化和数学最优化。我们的主要受控基准是递归固定探索，该方法使用相同的潜在发现情景和初始化，但在递归发现轮次中保持探索策略不变。此外，我们还与针对特定任务的领域基准进行比较。

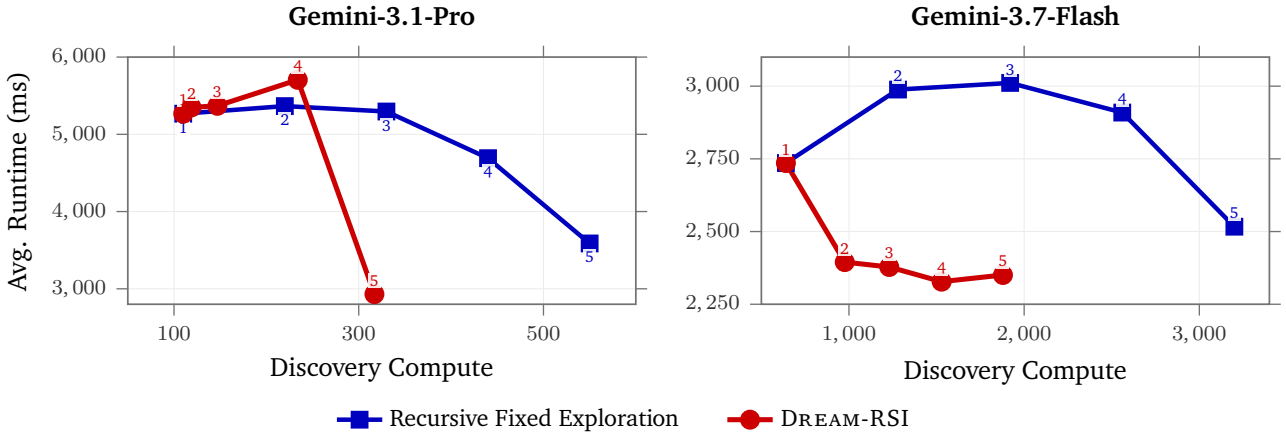
在所有任务中，Dream-RSI 和 Recursive Fixed Exploration 使用相同的发现智能体、评估器、初始化方式以及资源约束。两种方法均从同一手动设计的探索策略开始。该探索策略采用一种简单的 *并行精炼策略*：它并行启动多个独立的探索工作空间，每个工作空间维护其自身的本地发现轨迹，并基于该工作空间内累积的历史信息，反复精炼当前候选方案。因此，这两种方法在首个发现轮次中遵循相同的探索策略。在后续轮次中，Recursive Fixed Exploration 保持其探索策略不变，而 Dream-RSI 则通过在基于累积全局发现历史的回放模拟器上进行“梦境”推演，逐步精炼该策略，并在每一轮新发现中部署更新后的策略。发现成本以发现智能体调用的总累计次数来量化。

具体而言，我们通过 Gemini CLI 对 Gemini-3.1 Pro 和 Gemini-3.7-Flash 在多个递归发现轮次中进行评估。[2]. 在递归固定探索（Recursive Fixed Exploration）设置下，每轮中 Gemini-3.1 Pro 执行 10 个并行工作区，最多进行 11 次优化步骤（$10 \times 11 = 110$ discovery-agent 调用），而 Gemini-3.7-Flash 则执行 32 个并行工作区，最多进行 20 次优化步骤（$32 \times 20 = 640$ 调用）。Dream-RSI 保持每轮相同的预算，与基准在第 1 轮一致，同时在后续轮次中逐步更新其策略。关于递归轮次、任务设置、资源预算和评估协议的更多细节如下。

[2] https://geminicli.com/

Method	Model	Compute	*Non-biological*		*Biological*				Avg.
			Gisette	RCV1	DNA	Leukemia	Colon	Duke Breast	
Previous solvers									
sklearn	–	–	11275.2	252881.7	93.8	227.2	229.8	374.0	44180.3
glmnet	–	–	9063.6	73072.8	351.9	45.0	24.2	47.7	13767.5
SimpleTES	gpt-oss-120b	51,200	3141.9	19625.6	15.9	15.5	11.6	18.1	3804.8
SimpleTES †	gpt-oss-120b	51,200	8651.0	41143.1	37.6	28.2	19.5	31.1	8318.4
Our System									
Recursive Fixed Exploration	Gemini-3.1-Pro	550	1861.8	19550.1	41.5	26.1	14.5	28.4	3587.1
	Gemini-3.7-Flash	3200	1133.1	13873.0	29.8	24.1	15.7	24.4	2516.7
DREAM-RSI	Gemini-3.1-Pro	317	2841.0	14616.0	49.9	30.2	16.4	32.5	2931.0
	Gemini-3.7-Flash	1879	1091.9	12923.4	31.4	21.0	12.2	23.6	**2350.6**

(a) Final performance.



(b) Recursive Discovery Dynamics.

Figure 3 | **Lasso 正则化路径发现结果。 (a)** 六个保留的下游任务上的最终实际运行时间；越低越好。**计算**表示发现智能体调用的累计次数。**(b) 递归发现动态。** Gemini-3.1-Pro 与 Gemini-3.7-Flash 在六个保留下游任务上的平均下游运行时间，随发现计算量的累积变化情况。标记旁的数字表示递归轮次（迭代）。越低越好。

4.1. 算法工程

在此任务中，我们将 **Lasso 正则化路径**作为算法工程任务，这是高维统计中的一个基本计算原语，在基因组学和金融等领域的模型选择与交叉验证中被广泛使用。我们遵循 SimpleTES (Ye et al., 2026) 的基准情景，目标是发现完整的 Lasso 正则化路径的高效实现，同时保持数值正确性。在搜索过程中，我们使用与 SimpleTES 相同的 17 个合成实例，这些实例在维度、稀疏性、特征相关系数以及活动集结构方面涵盖了多样的问题情景。为了评估所发现的算法是否能泛化到搜索分布之外，我们还额外在六个保留的下游数据集上进行评估，这些数据集跨越了生物与非生物领域。

基准和设置。 我们与标准的 Lasso 求解器 **sklearn** (Pedregosa et al., 2011) 和 **glmnet** (Friedman et al., 2010) 进行对比，同时还包括 **SimpleTES** (Ye et al., 2026)，该方法使用 GPT-OSS-120B 并报

告了 51,200 次生成的预算。此外，我们还引入了递归固定探索作为受控基准。具体而言，我们运行递归固定探索和 DREAM-RSI 共 5 轮。

主要结果。 图 3(a) 总结了 Lasso 发现结果。在两种发现智能体骨干网络上，DREAM-RSI 均实现了优于递归固定探索的下游质量-计算权衡。使用 Gemini-3.1 Pro 时，它将六个保留数据集上的平均运行时间从 3587.1 ms 降低至 2931.0 ms，同时仅需 317 次发现智能体调用，而固定探索需要 550 次调用。使用 Gemini-3.7-Flash 时，DREAM-RSI 进一步将平均运行时间从 2516.7 ms 降至 2350.6 ms，仅需 1879 次调用，而固定探索则需 3200 次。尽管显著减少了发现计算开销，所得求解器在所有六个保留数据集上的表现仍优于标准的 sklearn 和 glmnet 实现。与使用 51,200 次生成的 SimpleTES 相比，DREAM-RSI 在发现智能体调用次数减少约两个数量级的情况下，实现了更低的平均下游运行时间。值得注意的是，Gemini-3.1-Pro 发现的程序特别适用于大规模矩阵（如 RCV1）。相比之下，Gemini-3.7-Flash 发现的程序更具通用性，在不同问题规模下均表现出稳定的性能。

递归发现动力学。 图 3(b) 展示了下游性能轨迹随递归发现轮次相对于累计发现计算量的变化。根据设计，两种方法在初始轮次中具有相同的搜索行为。在后续轮次中，递归固定探索保持静态的探索策略，而 DREAM-RSI 则通过对其积累的发现历史进行梦境模拟，逐步优化并重新部署其策略。因此，两条轨迹显著发散：DREAM-RSI 在整个过程中持续实现更优的下游性能，同时在 Gemini-3.1-Pro 和 Gemini-3.7-Flash 上所需的累计计算量大幅降低。

发现求解器分析。 我们进一步分析了所发现的求解器，其具体实现见附录 C。与根据问题维度在 LARS 和坐标下降之间切换的 SimpleTES 不同，所发现的求解器在活动集最优化过程中引入了自适应性。它将强规则筛选与基于柯西-施瓦茨不等式的 KKT 剪枝相结合，在边界无法确证某个特征时，仅选择性地重新计算确切梯度；当剪枝失效时，则退回到完整刷新。这种自适应验证机制进一步与高效的活动集管理、懒惰的格拉姆矩阵构造以及面向硬件的实现相结合。

4.2. 数学最优化

我们进一步在三个涵盖离散组合优化、几何最优化和函数最优化的数学发现任务上评估 DREAM-RSI：**和差问题**、**圆堆积**和 **自相关不等式**。这些任务的目标是在各自约束条件下，发现能够优化特定任务数学目标的高质量解。这三个任务的正式定义见附录。

我们使用通过 Gemini CLI 调用的 Gemini-3.1 Pro 作为递归固定探索和 DREAM-RSI 的发现智能体，进行 10 轮实验。对于每个任务，该智能体根据特定任务目标，迭代地提出并评估候选构造或最优化过程。我们与一系列现有的自动化发现系统进行了对比，包括 AlphaEvolve (Novikov et al., 2025)、AlphaEvolveV2 (Georgiev et al., 2025)、OpenEvolve (Sharma, 2025)、CodeEvolve (Assumpção et al., 2025)、ShinkaEvolve (Lange et al., 2026)、TTS-Discovery (Yuksekgonul et al., 2026)、ThetaEvolve (Wang et al., 2025)、EvoX (Liu et al., 2026a) 以及 SimpleTES (Ye et al., 2026)。

Table 1 | 数学发现任务的性能比较。对于 **Sum Diff** 和 **Circle Packing**，数值越高越好；而对于 **Auto Correlation**，数值越低越好。最佳结果以 **粗体**表示。

Method	LLM	Sum Diff (↑)	Auto Correlation (↓)	Circle Packing (↑)
AlphaEvolve	Gemini-2.0 Pro + Flash	–	1.455700	2.635862
AlphaEvolveV2	Gemini-2.0 Pro + Flash	1.121936	–	**2.635983**
OpenEvolve	-	–	1.460000	-
CodeEvolve	-	–	–	2.635980
ShinkaEvolve	Mixed	–	1.457800	2.635982
TTS-Discovery	Qwen3-8B	–	–	**2.635983**
ThetaEvolve	Distilled-Qwen3-8B	–	1.493000	**2.635983**
EvoX	Gemini-3.0-Pro	–	1.458900	2.635900
SimpleTES	GPT-OSS-120B	1.143975	**1.453675**	**2.635983**
Our System				
Recursive Fixed Exploration	Gemini-3.1-Pro	1.144047	1.456001	**2.635983**
Dream-RSI	Gemini-3.1-Pro	**1.145427**	1.456375	**2.635983**

结果。 表 1 总结了三个数学发现任务的结果。Dream-RSI 在求和-差分任务中取得了 1.145427 得分，优于 SimpleTES 和递归固定探索方法。在圆填充任务中，其得分达到 2.635983 ，与所比较方法中报告的最佳结果相当。对于自相关性任务，Dream-RSI 获得 1.456375 ，在现有发现系统中仍保持竞争力。值得注意的是，SimpleTES 在自相关不等式任务上达到了最先进水平，但需要 51,200 次生成，远高于我们方法所使用的少于 1,000 次生成。总体而言，这些结果表明，我们的 Dream-RSI 在数学最优化任务上具有良好的泛化能力。

4.3. 内核工程

我们进一步在 **GPU 内核工程**领域评估 Dream-RSI，其目标是在保持数值正确性的前提下，自动发现高性能的内核实现。与数学发现不同，内核工程需要同时对算法结构、内存访问、并行化以及硬件特定的最优化进行推理，为评估我们的 Dream-RSI 是否能够跨发现领域泛化提供了截然不同的测试平台。

我们从 KernelBench (Ouyang et al., 2025) 中考虑了四个典型的内核工程任务：**VGG16**、**LayerNorm**、**ConvDiv** 和 **ConvMax**。候选实现通过其执行性能进行评估，以反向运行时间（$1/\text{ms}$）为度量标准，并需通过与参考实现的正确性检查。我们使用 Gemini-3.1 Pro 作为编码智能体，并在相同的评估协议和初始化条件下，将 Dream-RSI 与递归固定探索（Recursive Fixed Exploration）进行比较。

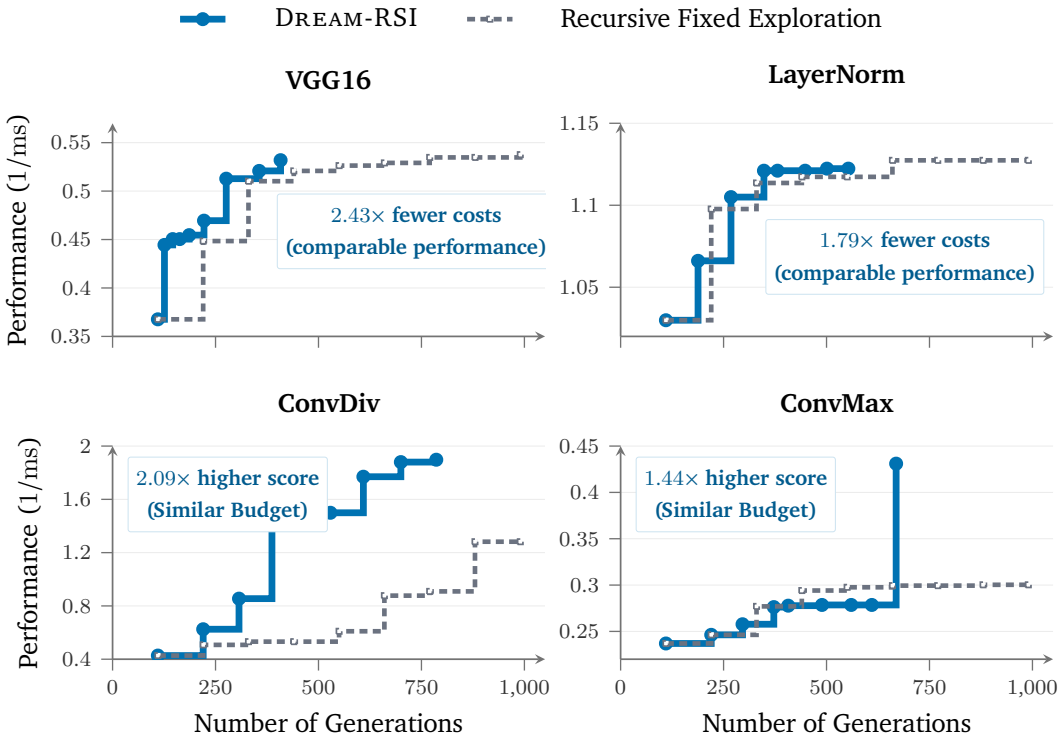


Figure 4 | **GPU 内核工程结果。** DREAM-RSI 与递归固定探索在不同代数下的发现性能表现。在 VGG16 和 LayerNorm 上，DREAM-RSI 分别比 2.43× 和 1.79× 少用若干代数即可达到相近性能。在 ConvDiv 和 ConvMax 上，其在相近的发现预算下分别实现了 2.09× 和 1.44× 的更高性能。所有任务中，数值越高越好。

结果。 图 4 展示了随着生成次数增加，发现轨迹的变化情况。在 VGG16 和 LayerNorm 上，DREAM-RSI 分别仅需 2.43× 和 1.79× 更少的生成次数即可达到相当的最终性能。在 ConvDiv 和 ConvMax 上，在相近的发现预算下，DREAM-RSI 分别实现了 2.09× 和 1.44× 的更高性能。这些结果表明，在递归轮次中调整探索策略可以提升长时程发现的效率与有效性。

5. 进一步分析

5.1. 长时程发现中的历史归纳偏置分析

我们进一步研究了历史归纳偏置的性质如何影响长时程发现。利用历史的一种自然替代方法是将先前的轨迹抽象为高层级的方向性洞察，并直接注入提示中，作为后续轮次的显式语义引导。为了评估这种提示级别的语义引导的有效性，我们将该方法应用于递归固定探索和DREAM-RSI。如图 5所示，在相同的发现预算下，这两种范式中的显式方向性引导的表现始

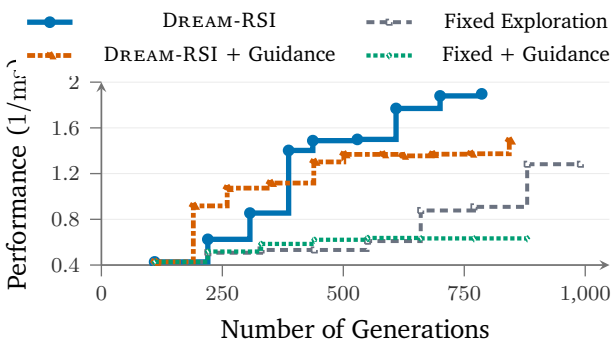


Figure 5 | ConvDiv 上的发现性能。将历史数据用作交互式回放模拟器，其效果优于仅将其用作指导。

终低于无引导的对应方法。这些结果表明，在长时程发现——其中部署了多个并行线程进行探索——的情境下，对未来的搜索方向施加强烈的语义归纳偏置往往会过度约束搜索空间，阻碍多样化的探索。

5.2. 探索行为演化的分析

图 6 展示了在 ConvDiv 上，学成探索策略如何在递归轮次中演化。如图所示，探索策略表现出明显的自适应模式：随着性能提升，其最初会节约发现计算量（例如，评估尝试次数从 110 减少到 50）。当进展随后趋于平缓时，它再次增加探索力度，这与进一步的性能提升相吻合。

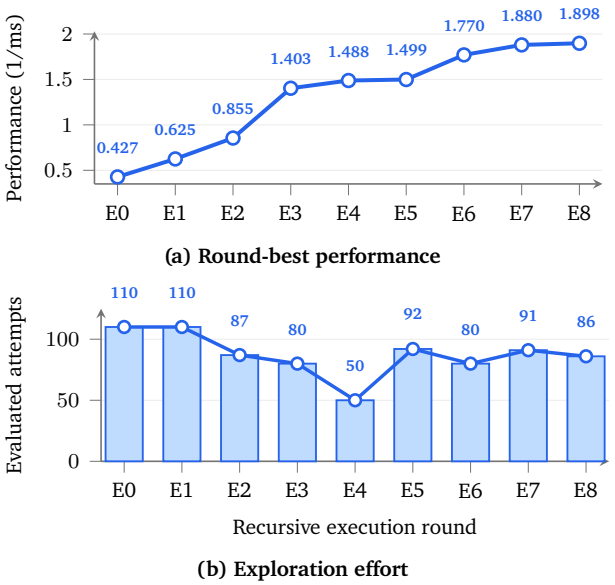


Figure 6 | **ConvDiv 上探索行为的演化。** **(a)** 在递归执行轮次中的每轮最佳性能。**(b)** 每轮中评估尝试的次数。

6. 相关工作

人工智能驱动的科学与算法发现。 基于大语言模型（LLM）的发现系统通过迭代生成、评估和优化候选解，利用先验成果与反馈信息，例如 AlphaEvolve (Novikov et al., 2025)、OpenEvolve (Sharma, 2025)、CodeEvolve (Assumpção et al., 2025)、ShinkaEvolve (Lange et al., 2026)、PACEvolve (Yan et al., 2026b)、DeltaEvolve (Jiang et al., 2026) 和 MLEvolve Du et al. (2026)。近期研究更加强调探索过程本身的重要性：SkyDiscover 提供了自适应发现基础设施 (Liu et al., 2026b)，SwarmResearch 动态编排多个搜索分支 (Virk et al., 2026)，而 EvoX (Liu et al., 2026a) 则显式地对搜索策略（而非仅对候选解）进行最优化。这一转变使探索本身成为一个元层级的优化问题；然而，针对探索策略的有效监督信号获取成本高昂且存在延迟，因为其质量往往只有在经历漫长的发现流程后才得以显现。

自我进化智能体。 更广泛的的研究方向探讨了智能体在与环境交互过程中自我改进其自身组件的方法。先前的方法通过测试时学习 (Wang et al., 2025; Wu et al., 2026a; Yan et al., 2026a; Yuksekgonul et al., 2026)，进化模型权重 (Huang et al., 2026a,c)、智能体利用方式 (Lee et al., 2026; Zhang et al., 2026b)、上下文 (Zhang et al., 2026d)、技能 (Ouyang et al., 2026a; Wu et al., 2026b; Zhang et al., 2026a)、模型行为、评估标准 (Xiong et al., 2026)、环境 (Huang et al., 2026b) 以及其他应用 (Dai et al., 2026)。大多数方法在 * 对象层面 * 进行操作，即改进用于任务执行或推理的组件。

近期工作开始优化元层面机制，包括搜索策略和自我改进过程 (Kim et al., 2026; Liu et al., 2026a; Wang et al., 2026; Yan et al., 2026a; Zhang et al., 2026c)。然而，这类元层面策略难以改进，因为其质量通常只有在代价高昂的长周期回放后才能显现。Dream-RSI 通过将累积的发现历史转化为回放模拟器，使这一元层面的最优化过程具有递归性和异策略特性，从而允许探索控制器反复被评估、改进并重新部署，而无需重复执行底层的发现过程。

记忆、历史与经验的复用。 先前的工作将智能体的经验重用为搜索历史、上下文、记忆、可复用技能或训练信号。DeltaEvolve 通过语义差异构建演化历史 (Jiang et al., 2026)；SwarmResearch 和 MLEvolve 利用跨分支或回溯信息来引导后续搜索 (Du et al., 2026; Virk et al., 2026)；其他工作则通过演化的上下文、更广泛的捕获状态、知识库或技能来改进智能体访问和保留经验的方式 (Lee et al., 2026; Ouyang et al., 2026a; Xu et al., 2026; Zhang et al., 2026d)。我们采取不同的视角：与其仅将探索历史用作下一次决策的上下文或记忆，我们将其组织为一个*回放模拟器*，使得多种替代的探索控制器能够被低成本评估。这将此前收集的发现经验转化为可用于元层级最优化的可复用反馈，缓解了用于改进探索策略的训练信号稀缺性和高成本问题。

7. 结论

我们提出了 Dream-RSI，一种用于递归自我改进中探索能力递归自我提升的框架。通过将累积的发现历史从静态上下文转化为可主动回放的模拟器，Dream-RSI 解决了元优化的核心瓶颈：延迟且昂贵的反馈，这在长时程发现情景中尤为严重。通过在基于历史发现树构建的回放模拟器中“做梦”，Dream-RSI 能够以极低的执行成本快速评估候选探索策略。随后，改进后的策略被重新部署到线上，以推动进一步发现并扩展模拟器池，从而完成递归自我改进环。在算法工程、数学优化和 GPU 内核工程等多个领域，Dream-RSI 在多个设置下均实现了具有竞争力或更优的发现质量，同时显著降低了发现成本。

References

Anthropic. Learning more about claude's mathematical capabilities. https://www.anthropic.com/research/riemann-zeta, Aug. 2026. Accessed: 2026-08-13.

H. Assumpção, D. Ferreira, L. Campos, and F. Murai. Codeevolve: an open source evolutionary coding agent for algorithmic discovery and optimization. *arXiv preprint arXiv:2510.14150*, 2025.

S. Cao, Z. Mao, J. E. Gonzalez, and I. Stoica. K-search: Llm kernel generation via co-evolving intrinsic world model. *arXiv preprint arXiv:2602.19128*, 2026.

R. Dai, K. Huang, C. Kang, and C. Liao. It takes two to match: Co-evolving generative retriever with reinforcement learning. *arXiv preprint arXiv:2609.00638*, 2026.

S. Du, X. Yan, J. Shi, Z. Cao, S. Feng, Z. Liang, B. Sun, T. Peng, Y. Zhou, X. Li, et al. Mlevolve: A self-evolving framework for automated machine learning algorithm discovery. *arXiv preprint arXiv:2606.06473*, 2026.

J. H. Friedman, T. Hastie, and R. Tibshirani. Regularization paths for generalized linear models via coordinate descent. *Journal of statistical software*, 33:1–22, 2010.

B. Georgiev, J. Gómez-Serrano, T. Tao, and A. Z. Wagner. Mathematical exploration and discovery at scale. *arXiv preprint arXiv:2511.02864*, 2025.

S. Gupta, J. Davidson, S. Levine, R. Sukthankar, and J. Malik. Cognitive mapping and planning for visual navigation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2616–2625, 2017.

D. Ha and J. Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2(3):440, 2018.

D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019.

D. Hafner, T. Lillicrap, M. Norouzi, and J. Ba. Mastering atari with discrete world models. *arXiv preprint arXiv:2010.02193*, 2020.

D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

D. Hafner, W. Yan, and T. Lillicrap. Training agents inside of scalable world models. *arXiv preprint arXiv:2509.24527*, 2025.

Y. Hu, S. Liu, Y. Yue, G. Zhang, B. Liu, F. Zhu, J. Lin, H. Guo, S. Dou, Z. Xi, et al. Memory in the age of ai agents. *arXiv preprint arXiv:2512.13564*, 2025.

C. Huang, H. Liu, T. Zheng, R. Dai, L. Huang, J. Li, Z. Li, Z. Wei, Y. Meng, and J. Huang. G-zero: Self-play for open-ended generation from zero data. *arXiv preprint arXiv:2605.09959*, 2026a.

C. Huang, Z. Wang, R. Han, J. Yan, Y. Chen, Z. CuiZhu, K. Jiang, P. Xia, H. Yu, Y. Zhuang, et al. Envharness: Awakening static worlds for agent learning. *arXiv preprint arXiv:2608.19880*, 2026b.

C. Huang, W. Yu, X. Wang, H. Zhang, Z. Li, R. Li, J. Huang, H. Mi, and D. Yu. R-zero: Self-evolving reasoning llm from zero data. In *International Conference on Learning Representations*, volume 2026, pages 130770–130790, 2026c.

J. Jaber and O. Jaber. Autokernel: Autonomous gpu kernel optimization via iterative agent-driven search. *arXiv preprint arXiv:2603.21331*, 2026.

J. Jiang, T. Ding, and Z. Zhu. Deltaevolve: Accelerating scientific discovery through momentum-driven evolution. *arXiv preprint arXiv:2602.02919*, 2026.

Z. M. Kim, Y.-J. Lee, S. Jwa, and D. Kang. Meta^n: Recursive self-improvement through emergent depth. *arXiv preprint arXiv:2608.24735*, 2026.

R. Lange, Y. Imajuku, and E. Cetin. Shinkaevolve: Towards open-ended and sample-efficient program evolution. In *International Conference on Learning Representations*, volume 2026, pages 74026–74078, 2026.

Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab, and C. Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.

S. Liu, S. Agarwal, M. Maheswaran, M. Cemri, Z. Li, Q. Mang, A. Naren, E. Boneh, A. Cheng, M. Z. Pan, et al. Evox: Meta-evolution for automated discovery. *arXiv preprint arXiv:2602.23413*, 2026a.

S. Liu, M. Cemri, S. Agarwal, A. Krentsel, A. Naren, Q. Mang, Z. Li, A. Gupta, M. Maheswaran, A. Cheng, M. Pan, E. Boneh, K. Ramchandran, K. Sen, M. Zaharia, A. G. Dimakis, and I. Stoica. Skydiscover: A flexible, adaptive framework for ai-driven scientific and algorithmic discovery. In *Proceedings of the ACM Conference on AI and Agentic Systems*, CAIS '26, pages 1223–1227. Association for Computing Machinery, 2026b. doi: 10.1145/3786335.3813221. URL https://doi.org/10.1145/3786335.3813221.

S. Liu, Z. Lin, Y. Zhang, Y. Ren, Y. Wu, Y. Li, Z. Wang, Z. Fu, and J. Ye. The path to recursive self-improving agents: Foundation, framework, and future directions. *Preprints*, August 2026c. doi: 10.20944/preprints202608.0051.v1. URL https://doi.org/10.20944/preprints202608.0051.v1.

T. M. Moerland, J. Broekens, A. Plaat, and C. M. Jonker. Model-based reinforcement learning: A survey. *Foundations and Trends in Machine Learning*, 16(1):1–118, 2023.

A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. Ruiz, A. Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

OpenAI. On the navier–stokes millennium prize problem. https://openai.com/index/navier-stokes-solution/, Sept. 2026. Accessed: 2026-09-10.

A. Ouyang, S. Guo, S. Arora, A. L. Zhang, W. Hu, C. Ré, and A. Mirhoseini. Kernelbench: Can llms write efficient gpu kernels? *arXiv preprint arXiv:2502.10517*, 2025.

S. Ouyang, J. Yan, Y. Chen, R. Han, Z. Wang, B. D. Mishra, R. Meng, C.-L. Li, Y. Jiao, K. Zha, et al. Skillos: Learning skill curation for self-evolving agents. *arXiv preprint arXiv:2605.06614*, 2026a.

S. Ouyang, J. Yan, I. Hsu, Y. Chen, K. Jiang, Z. Wang, R. Han, L. Le, S. Daruki, X. Tang, et al. Reasoningbank: Scaling agent self-evolving with reasoning memory. In *International Conference on Learning Representations*, volume 2026, pages 94327–94354, 2026b.

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al. Scikit-learn: Machine learning in python. *the Journal of machine Learning research*, 12:2825–2830, 2011.

B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.

A. Sharma. Openevolve: an open-source evolutionary coding agent, 2025. URL https://github.com/algorithmicsuperintelligence/openevolve.

R. S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In B. Porter and R. Mooney, editors, *Machine Learning Proceedings 1990*, pages 216–224. Morgan Kaufmann, San Francisco (CA), 1990. ISBN 978-1-55860-141-3. doi: https://doi.org/10.1016/B978-1-55860-141-3.50030-4. URL https://www.sciencedirect.com/science/article/pii/B9781558601413500304.

Y. Virk, Z. Edds, C. S. Xia, and L. Zhang. Swarmresearch: Orchestrating coding agents for open-ended discovery. *arXiv preprint arXiv:2607.02807*, 2026.

Y. Wang, S.-R. Su, Z. Zeng, E. Xu, L. Ren, X. Yang, Z. Huang, X. He, L. Ma, B. Peng, et al. Thetaevolve: Test-time learning on open problems. *arXiv preprint arXiv:2511.23473*, 2025.

Z. Wang, M. Yan, J. Bi, S. Yan, V. Tresp, and Y. Ma. Metaskill-evolve: Recursive self-improvement of llm agents via two-timescale meta-skill evolution. *arXiv preprint arXiv:2607.05297*, 2026.

S. Wu, C. Qian, X. Chen, and H. Ji. Teaching llms to self-evolve: Cultivating core meta-skills with reinforcement learning. *arXiv preprint arXiv:2607.21971*, 2026a.

X. Wu, Y. Zhuan, R. Wei, H. Chen, D. Bai, J. Liu, X. Wang, X. Wang, L. Wang, and X. Cheng. Agenticrectune: Multi-agent with self-evolving skillhub for recommendation system optimization. *arXiv preprint arXiv:2604.26969*, 2026b.

T. Xiong, Z. Yang, X. Wang, C.-C. Lin, R. Ma, K. Lin, Z. Wang, L. Li, C. Liu, R. Chen, et al. Rubrics as visual-repair context for self-evolving ui-to-code generation. *arXiv preprint arXiv:2608.24138*, 2026.

W. Xu, A. Sordoni, C. Singh, Z. Gero, M. Galley, X. Yuan, and J. Gao. Test-time learning with an evolving library. *arXiv preprint arXiv:2605.14477*, 2026.

M. Yan, B. Peng, B. Coleman, Z. Chen, Z. Xie, S. Chen, Z. He, N. Sachdeva, W. Wang, E. H. Chi, et al. Pacevolve++: Improving test-time learning for evolutionary search agents. *arXiv preprint arXiv:2605.07039*, 2026a.

M. Yan, B. Peng, B. Coleman, Z. Chen, Z. Xie, S. Chen, Z. He, N. Sachdeva, I. Ye, W. Wang, et al. Pacevolve: Enabling long-horizon progress-aware consistent evolution. *arXiv preprint arXiv:2601.10657*, 2026b.

H. Ye, H. Lin, J. Tang, Y. Luo, C. Yang, C. Su, R. Thapa, R. Yang, R. Liu, Z. Li, et al. Evaluation-driven scaling for scientific discovery. *arXiv preprint arXiv:2604.19341*, 2026.

M. Yuksekgonul, D. Koceja, X. Li, F. Bianchi, J. McCaleb, X. Wang, J. Kautz, Y. Choi, J. Zou, C. Guestrin, et al. Learning to discover at test time. *arXiv preprint arXiv:2601.16175*, 2026.

H. Zhang, S. Fan, H. P. Zou, Y. Chen, Z. Wang, J. Zhou, C. Li, W.-C. Huang, Y. Yao, K. Zheng, et al. Coevoskills: Self-evolving agent skills via co-evolutionary verification. *arXiv preprint arXiv:2604.01687*, 2026a.

J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune. Darwin gödel machine: open-ended evolution of self-improving agents. In *International Conference on Learning Representations*, volume 2026, pages 104223–104294, 2026b.

J. Zhang, B. Zhao, W. Yang, J. Foerster, J. Clune, M. Jiang, S. Devlin, and T. Shavrina. Hyperagents. *arXiv preprint arXiv:2603.19461*, 2026c.

Q. Zhang, C. Hu, S. Upasani, B. Ma, F. Hong, V. Kamanuru, J. Rainton, C. Wu, M. Ji, H. Li, et al. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*, volume 2026, pages 86069–86100, 2026d.

T. Zheng, H. Liu, C. Huang, H. Bao, S. Zhang, R. Liu, R. Dai, R. Chen, C. Liu, T. Xiong, et al. Llms improving llms: Agentic discovery for test-time scaling. *arXiv preprint arXiv:2605.08083*, 2026a.

T. Zheng, H. Zhang, W. Yu, X. Wang, H. Xing, R. Dai, R. Liu, H. Bao, C. Huang, H. Huang, et al. Parallel-r1: Towards parallel thinking via reinforcement learning. In *International Conference on Learning Representations*, volume 2026, pages 121144–121166, 2026b.

A. 详细任务描述

Problem 1 (Lasso Regularization Path)

给定一个特征矩阵 $X \in \mathbb{R}^{n\times p}$ ，响应 $y \in \mathbb{R}^n$ ，以及一个递减序列 $\lambda_1 > \cdots > \lambda_K$ ，定义

$$F_k(w) := \frac{1}{2n}\|y - Xw\|_2^2 + \lambda_k\|w\|_1, \qquad w_k^\star \in \arg\min_{w\in\mathbb{R}^p} F_k(w).$$

候选求解器返回近似系数 $\widetilde{W} = (\widetilde{w}_1, \ldots, \widetilde{w}_K)$ 。若其通过基准的目标值检验，则

$$F_k(\widetilde{w}_k) \le F_k(w_{k,\mathrm{sklearn}}) + 10^{-6}, \qquad \text{for every } k,$$

正确性在与计时实例不同的新实例上进行检查。如果任何必要的检查失败，则搜索得分为零。
否则，令 $\mathcal{I}$ 表示时间实例，t_i 表示在实例 i 上计算完整正则化路径所需的时间，则搜索得分为

$$R_{\mathrm{search}} = \left(\prod_{i\in\mathcal{I}} t_i\right)^{-1/|\mathcal{I}|}.$$

Problem 2 (Sum–Difference Problem)

和–差问题要求一个有限集 $A \subset \mathbb{Z}$，其规范化的和集相对于其规范化的差集要足够大。目标是

$$\Gamma(A) := \frac{\log(|A + A|/|A|)}{\log(|A - A|/|A|)},$$

在哪里

$$A + A := \{a + a' : a, a' \in A\}, \qquad A - A := \{a - a' : a, a' \in A\}.$$

目标是构造一个有限集 $A \subset \mathbb{Z}$，使其最大化 $\Gamma(A)$ 。

Problem 3 (Circle Packing in a Unit Square)

对于 $n \in \{26, 32\}$，圆填充任务要求确定中心 $(x_i, y_i) \in [0, 1]^2$ 和半径 $r_i \ge 0$，使得每个圆都位于单位方阵内且任意两个圆不重叠：

$$r_i \le x_i \le 1 - r_i, \qquad r_i \le y_i \le 1 - r_i,$$

$$(x_i - x_j)^2 + (y_i - y_j)^2 \ge (r_i + r_j)^2, \qquad 1 \le i < j \le n.$$

目标是最大化

$$\sum_i r_i.$$

Problem 4 (Autocorrelation Inequalities)

对于一个可积函数 $f : \mathbb{R} \to \mathbb{R}$，定义其自卷积

$$(f * f)(t) := \int_{\mathbb{R}} f(t-x)f(x)\,dx, \qquad t \in \left[-\frac{1}{2}, \frac{1}{2}\right].$$

第一自相关不等式。寻找一个在 $\left[-\frac{1}{4}, \frac{1}{4}\right]$ 上定义的非负可积函数 $f : \mathbb{R} \to \mathbb{R}$，使得

$$\int_{-1/4}^{1/4} f(x)\,dx = 1.$$

目标是使最小化

$$\Phi_1(f) := \max_{t \in [-1/2, 1/2]} (f * f)(t).$$

第二自相关不等式。找出一个在 $\left[-\frac{1}{4}, \frac{1}{4}\right]$ 上非负可积的函数 $f : \mathbb{R} \to \mathbb{R}$，使得

$$\int_{-1/4}^{1/4} f(x)\,dx = 1.$$

目标是最大化

$$\Phi_2(f) := \frac{\|f * f\|_2^2}{\|f * f\|_1 \|f * f\|_\infty}.$$

第三个自相关不等式。找出一个在 $\left[-\frac{1}{4}, \frac{1}{4}\right]$ 上定义的可积（可能为带符号的）函数 $f : \mathbb{R} \to \mathbb{R}$，使得

$$\int_{-1/4}^{1/4} f(x)\,dx = 1.$$

目标是使最小化

$$\Phi_3(f) := \max_{t \in [-1/2, 1/2]} |(f * f)(t)|.$$

B. 提示词

为保证可复现性，我们提供了在线探索和基于回放的探索-策略改进所使用的提示。用美元符号或花括号包围的变量将在运行时系统执行前进行实例化。

B.1. 探索提示

以下提示用于在在线探索过程中引导发现智能体。它要求智能体在提出新解决方案之前，检查完整的可用发现历史，明确推理成功与失败的尝试，并避免反复利用局部饱和的方向。

Listing 1 | Prompt used for online exploration.

```
You must read every historical proposal before proposing or implementing a new solution.

$direction_guidance

Variables (‘$node_dir‘, ‘$history_dir‘, ‘$baseline_dir‘, ‘$eval_program‘, ‘$problem_file‘) are filled in by the calling system.
    ‘$node_dir‘ is your own attempt directory — exclude it when scanning sibling ‘attempt_*/‘ dirs.

## 1. Read the complete history first
```

```
Before proposing anything, read every ‘proposal.md‘ under sibling ‘attempt_*/‘ dirs, ‘$history_dir‘, and ‘$baseline_dir‘ in full — not a sample, not just recent cycles or the current branch. For each, read its matching ‘eval/score.json‘ (and ‘error.txt‘ if it failed). Trust the measured result over what the proposal claims about itself.

## 2. Learn from both successes and failures

For every past attempt, note the mechanism and how it did. For failures, figure out *why*: a flawed core idea, or a good idea let down by a bug, bad parameters, or an implementation slip? Don’t repeat the former. The latter is worth retrying — but only once you’ve actually located the bug in the code (not just guessed from the proposal), and only with a specific fix in hand.

## 3. Don’t converge into a local optimum

Look at the shape of what’s been tried. If most attempts cluster around small variations of one mechanism with flattening returns, that’s a local optimum - resist proposing another small tweak there. Deliberately favor a structurally different mechanism or an untried combination over a safer marginal refinement. Exploration diversity matters as much as the next incremental gain.

## 4. Propose and implement

The new idea must be a genuinely new mechanism, a new combination of previously-successful pieces, or a targeted fix to a specific bug found in step 2 - never a repeat or rename of something already tried. Implement it in ‘$eval_program‘. Don’t claim it compiles, is correct, or beats SOTA until it’s actually evaluated.

## Files

Write only ‘$node_dir/proposal.md‘ (mechanism, evidence from history, why it’s not a repeat, expected benefit/risk) and ‘$node_dir/$eval_program‘. Everything else is read-only.

## Note:
   Never execute pkill, kill, killall, or terminate unrelated processes.
```

B.2. 基于回放的策略改进提示

控制器开发智能体在历史回放过程中使用以下提示。该智能体根据先前收集的发现轨迹回放所获得的反馈，修改探索策略，同时仍受限于前缀可观察信息。

Listing 2 | Prompt used for replay-based improvement of the exploration policy.

```
You are improving one **prefix-only exploration policy**. Edit only
‘‘{method_file}‘‘ and implement ‘‘OptimalPolicy.solve(self, question, budget=None)‘‘.
Do not solve the scientific task and do not edit any other program.

## Objective: quality, work, and parallelism

The environment is a frozen, irregular branch×attempt grid. A policy opens a root
or refines the next cell of an already-open branch. Each revealed cell costs one
probe. The policy sees only the cells it has revealed so far; unrevealed scores are
unknown.

The evaluator sweeps your single ‘‘beta‘‘ knob and ranks the resulting curve by:

```

```
    pareto.reward = pareto.auc - lambda * parallel_penalty

``pareto.auc`` rewards reaching high per-trace attainment with few **total probes**.
``parallel_penalty`` is the mean of
``effective_sequential_rounds / total_probes`` over the sweep. For a batch of size
``k`` with ``W = question.max_parallelism`` workers, it costs one decision round and
``ceil(k / W)`` effective sequential rounds. A serial policy has penalty near 1;
useful full batches approach ``1/W``. Therefore choose only promising probes, but
batch independent promising probes whenever possible.

A local implementation failure does not by itself prove that its parent direction
is poor. Weigh recovery value against new roots and ordinary refinements while
keeping batches parallel.

## API

    question.reset()
    question.observed() -> dict[str, Observation]   # revealed prefix only
    question.legal_actions() -> list[str]           # roots + opened-branch frontiers
    question.legal_roots() -> list[str]             # unopened roots only
    question.opened_branches() -> list[int]
    question.meta(cell_id) -> CellMeta              # .branch .attempt .parent_id .seq .tags
    question.probe_batch(cells, on_reveal=...) -> list[Observation]
    question.baseline_score
    question.max_parallelism

``Observation`` supplies ``branch``, ``attempt``, ``score``, ``evaluated``, ``valid``,
``fail_class``, ``error``, ``delta_vs_baseline``, ``delta_vs_parent``, ``n_valid``, and
``n_total``.
Use the helpers in ``see.policy.observation_signal`` when useful:
``branch_promising``, ``branch_failed_hard``, ``probe_improved_vs_parent``, and
``probe_improved_vs_baseline``.

**Success semantics:** an evaluated observation with ``error is None`` and
``fail_class == "ok"`` is a successful evaluation, even when ``valid == False`` or
``n_valid``/``n_total`` are unavailable. Never label it repairable solely because
``valid`` is false. A *successful anchor* below means the best historical score
from such a successful evaluation.

Do **not** use ``question.best_so_far`` or ``question.budget_spent`` to decide what
to explore; they are bookkeeping only. Derive any decision statistic from
``question.observed()`` instead.

## Required branch trajectory and failure interpretation

For each opened branch, reconstruct its ordered prefix trajectory, not only its
latest observation or best score: successful anchor, score trend, regressions,
failure/repair sequence, and explored versus remaining depth.

Before closing or deprioritizing a failed frontier, classify it as
hard-unrecoverable, repairable implementation failure, weak-but-underexplored, or
repeatedly unpromising after sufficient valid evidence. Output/correctness mismatch,
shared-memory/resource limits, and variable/code, mask/layout/shape errors are
normally repairable. Do not infer algorithmic failure from one such error.
```

68 ‘‘n_valid == 0‘‘ and ‘‘branch_failed_hard(obs)‘‘ are signals, not unconditional
69 closure: use ‘‘fail_class‘‘ and ‘‘error‘‘ to distinguish a repairable zero-valid
70 failure from an environment/dependency failure. ‘‘compile_other‘‘ alone is not
71 permanently hard. Classify the current failure episode: a later successful result
72 reopens the branch and cancels closure based only on an earlier failure.
73
74 ## Required batch decision loop
75
76 At each decision round:
77
78 1. Read the prefix, reconstruct trajectories, and close only branches with
79 cumulative evidence of being hard-unrecoverable or repeatedly unpromising.
80 2. Rank legal roots and legal branch frontiers using only prefix-derived signals:
81 successful anchor, parent→child gain, complete branch trajectory, actual success
82 versus failure evidence,
83 failure recoverability, prior repair outcomes, remaining depth, and cross-branch
84 comparison.
85 3. Rank actual repairable failures and underexplored frontiers in deterministic
86 queues using trajectory, recoverability, remaining depth, repeated failures, and
87 beta. A repairable failure retains eligibility unless cumulative evidence lowers
88 its relative priority.
89 4. Build one **dynamic portfolio** batch of independent candidates, up to
90 ‘‘question.max_parallelism‘‘: exploitation (strong normal refinements),
91 exploration (new roots or underexplored branches), and at most one recovery
92 (an actual repairable failure). When multiple roles are eligible, give
93 exploration and justified recovery representation before filling remaining slots
94 by priority; adapt this to prefix evidence rather than fixed quotas. Recovery
95 must not displace normal successful refinements or leave workers idle. Never
96 sample randomly, and do not default to a singleton merely because its top
97 candidate is clear.
98 5. Stop only after considering the whole revealed portfolio: active, underexplored,
99 recoverable, unopened, and remaining legal candidates. Do not stop while an
100 eligible high-priority recovery or underexplored candidate remains; every
101 remaining action needs an evidence-based decision to continue, reserve, or close.
102
103 A batch must contain distinct cells that are all legal *before* the call. It may
104 contain several roots and/or one frontier from each opened branch. It must never
105 contain a parent and its child together. Do not use a fixed widen-all / deepen-all
106 wave schedule: adapt batch composition after every revealed prefix.
107
108 Minimal structure:
109
110 ```
111 from see.policy.api import (
112 LLMDesignedMethod, SimResult, _budget_done, _record_curve, finalize_result,
113)
114
115 def solve(self, question, budget=None):
116 question.reset()
117 res, closed = SimResult(), set()
118 while not _budget_done(question, budget):
119 prefix = question.observed()
120 update_closed(closed, prefix, question)
121 batch = select_batch(prefix, question, closed)
122 if not batch:
```
```

```
                break
            question.probe_batch(
                batch,
                on_reveal=lambda _: _record_curve(res, question),
            )
        return finalize_result(question, res)

## Hard constraints

- Keep ``NAME = "OptimalPolicy"`` and implement
  ``class OptimalPolicy(LLMDesignedMethod)`` in ``{method_file}`` only.
- **Prefix-only:** decisions may use revealed observations, ``baseline_score``, legal
  sets, structural ``meta``, and helper signals. Never use unrevealed scores, a true
  optimum, hardcoded winning cell ids, absolute score targets, or internal trace data.
- Every prune, widen, deepen, batch, and stop decision must be explainable from the
  current prefix. Shallow weak scores are not enough to discard a branch: deeper
  attempts can recover. A repairable latest failure must not erase its historical
  successful anchor or by itself cause permanent starvation.
- Replay calls with ``budget=None``. Always terminate when no batch is selected; do
  not assume a budget cap exists.
- A selected batch must be legal, have no duplicate ids, and contain at most
  ``question.max_parallelism`` cells.

## Beta: fixed per run, adaptive across cycles

Read exactly one scalar in ``___init___``:

    beta = float(self.config.get("beta", <sensible_default>))

Beta has three distinct roles. Do not conflate them:

1. **Within one replay or live episode:** beta is fixed. Route every behavioral
   threshold through one ``_schedule(beta) -> dict``. High beta means more width,
   deeper patience, and weaker pruning. Low beta means fewer probes, earlier
   stagnation stops, and stronger pruning. Never change beta from observations inside
   ``solve()``. Route recovery eligibility, reserve threshold, and waiting through
   the same schedule: high beta is more patient; low beta remains selective without
   treating one repairable failure as automatic closure.
2. **During offline evaluation:** eval sweeps a fixed beta grid. This measures whether
   the policy exposes a real attainment/work/parallelism trade-off; it is not online
   beta adaptation.
3. **When proposing the next policy version:** choose the baked-in default beta once,
   using evidence from earlier *live* cycles and their beta sweeps. That default will
   remain fixed throughout the next live exploration episode.

Keep all thresholds relative to the prefix; never use absolute score cutoffs.

Use the following cross-cycle default-beta rule. Read the most recent 2–3
**live** ``trace_pool/iter*/live_cycle_manifest.json`` sidecars (and ``_current``
when present) for each iteration's final best score and actual baked-in beta. Read
the matching archived ``beta_sweep.json`` values (``pareto.reward``, AUC, parallel
penalty, and the per-beta frontier). Scores alone do not establish that beta caused a
change, so always use both sources:

```

- live best is still improving: keep the prior default beta unless its sweep clearly shows a better nearby beta;
- live best has plateaued, and higher beta reaches higher attainment for a reasonable work/parallelism cost in the sweep: raise the default by a small step (about 0.1–0.2, clamped to [0, 1]);
- a high default beta has already been tried through a plateau, and high-beta sweep points add work without higher attainment: lower it by a small step;
- history is insufficient or evidence conflicts: use a moderately exploratory default (about 0.6), rather than pretending the replay ceiling is a live stopping signal.

The beta sweep is non-degenerate only if beta changes the attainment/work trade-off. It also reveals whether the policy batches. Do not select the default simply as the smallest beta that reaches a frozen trace's known ceiling.

Required next-cycle grid planning

Every proposed policy **must** implement this deterministic method:

```
    from see.policy.api import GridPlan, GridPlanningContext

    def plan_grid(self, context: GridPlanningContext) -> GridPlan:
        ...
```

This method runs **before** a new live grid is created. It does not make a within-episode decision and must never inspect a current episode's outcomes. It must always return a non-``None`` ``GridPlan``: do not inherit the template stub and do not delegate grid choice to the runner's fallback. When history is empty or insufficient, still return an explicit conservative bootstrap plan derived from the context's fallback/hard-cap fields, with a factual reason.

``GridPlan(branch_count=W, refine_count=R)`` accepts arbitrary integers, not a fixed set of presets. It creates branches ``0..W-1`` and attempts ``0..R``; ``R`` is the number of refinements allowed after each root. The runner validates ``1 <= W <= context.hard_max_branch_count`` and ``0 <= R <= context.hard_max_refine_count``. In replay, a requested plan beyond the frozen trace's ``context.trace_branch_count`` or ``context.trace_refine_count`` is out of support and cannot earn replay reward.

Use only the prefix-safe facts in ``context``:

- ``history``: completed earlier live manifests, including prior planned/effective grids, actual opened width/depth, probe work, decision rounds, scores, and beta;
- fallback/hard caps and worker cap;
- replay structural support fields. Do not read raw trace outcomes or a current cycle result inside ``plan_grid``.

Choose width versus depth from evidence, not a default preference:

- many semantically distinct roots improve early while deeper refinements stall: increase width and reduce/hold depth;
- high gains arrive late on a small, repeatable set of directions: reduce/hold width and increase depth;
- all explored directions plateau after sufficient depth while meaningful direction classes remain uncovered: increase width;

```
- repeated hard, unrecoverable failures or strongly redundant directions: reduce
  width and depth conservatively;
- conflicting or insufficient history: return an explicit conservative bootstrap
  plan derived from the context, and state that evidence is insufficient.

Include a short, factual ‘‘reason‘‘ in every plan. ‘‘plan_grid‘‘ answers
how many directions to make available; the direction provider assigns those new
roots their directions, and ‘‘solve‘‘ still decides which legal roots/frontiers to
open, refine, prune, or stop. Do not choose roots merely because their branch id is
small. The runtime grid is the hard bound: controller thresholds may use less, but
can never create branches or attempts beyond the effective plan. Before finishing,
verify that the edited ‘‘method.py‘‘ contains an override of ‘‘plan_grid‘‘ that
returns ‘‘GridPlan(branch_count=..., refine_count=..., reason=...)‘‘ on every path.

## Learn from history without leaking outcomes

Earlier rounds are in ‘‘{history_dir}/r####_*/‘‘. Read their policy code and
‘‘proposal_results/beta_sweep.json‘‘. Start from a strong recent policy, retain
mechanisms that raised ‘‘pareto.reward‘‘, and make a concrete change when progress
stalls. A legacy AUC-only sweep is useful code history but is not numerically
comparable to the current reward. The baseline under ‘‘{history_dir}/baseline/‘‘ is
a parallel-refine floor to beat.

Each current-objective round also archives
‘‘proposal_results/policy_execution_traces.jsonl‘‘: one replay episode per
‘‘(frozen trace, beta)‘‘. Use it to diagnose general behavior — serial batches,
premature stops, over-pruning, or wasted probes — from the prefix state, selected
batch, and revealed outcomes at each decision round. It is **between-round feedback
only**: never read it inside ‘‘solve()‘‘, and never copy a trace-specific branch,
cell id, score, or target into policy logic.

‘‘{trace_pool}‘‘, if present, may be read only outside ‘‘solve()‘‘. Prefer the
‘‘live_cycle_manifest.json‘‘ sidecars over raw replay outcomes for the per-iteration
live trend. Never copy trace scores, targets, or cell ids into policy logic.

## Deliverable

Write a complete adaptive policy in ‘‘{method_file}‘‘. Include a short module
docstring describing its prefix signals, batch rule, beta schedule, default-beta
rationale, grid-planning rule (if implemented), and safeguards against
over-pruning, over-stopping, permanent starvation after repairable failures, and
serial probes. Before finishing, verify trajectory-based ranking, the stated
success semantics, non-automatic zero-valid closure, deterministic recovery
competition, and portfolio-level stop.
```

C. 发现的程序

我们提供了 Dream-RSI 发现的 Lasso 路径求解器的完整实现。如第 4.1 节所述，该求解器结合了强规则筛选、自适应 Cauchy–Schwarz KKT 剪枝、不相交的活动集记录、延迟的 Gram 矩阵构造以及硬件感知最优化。

Listing 3 | Complete Lasso-path solver discovered by Dream-RSI.

```
# EVOLVE-BLOCK-START

CPP_CODE = r'''
#define EIGEN_NO_DEBUG
#define EIGEN_MPL2_ONLY
#define EIGEN_UNROLL_LOOPS

#include <Eigen/Dense>
#include <vector>
#include <cstdio>
#include <cmath>
#include <algorithm>
#include <numeric>
#include <omp.h>
#include <cstdlib>
#include <cstring>

using Eigen::MatrixXd;
using Eigen::VectorXd;

#if defined(_MSC_VER)
#define RESTRICT __restrict
#elif defined(__GNUC__) || defined(__clang__)
#define RESTRICT __restrict__
#else
#define RESTRICT
#endif

// High-performance alignment assumption
#if defined(__GNUC__) || defined(__clang__)
#define ASSUME_ALIGNED(ptr, alignment) (double*)__builtin_assume_aligned((ptr), (alignment))
#else
#define ASSUME_ALIGNED(ptr, alignment) (ptr)
#endif

// High-performance branch-free soft-thresholding using std::abs and std::copysign
static inline double soft_thresh(double z, double gamma) {
    double abs_z = std::abs(z);
    double val = abs_z - gamma;
    return std::copysign(val > 0.0 ? val : 0.0, z);
}

//
    ============================================================================

// DISJOINT-PARTITION ACTIVE-SET LASSO PATH SOLVER WITH ALIGNED COLUMN PADDING
//
    ============================================================================

void solve_active_set(
    const double* RESTRICT X_padded,
    int n_padded,
    int n,
```

```
    int p,
    const VectorXd& y,
    const VectorXd& lam_path,
    const VectorXd& xv,
    const VectorXd& grad_init,
    MatrixXd&       coef_path,  // (p, n_lam) output, pre-zeroed
    double thresh,              // convergence threshold
    int    maxit)               // max inner loop iterations
{
    const double fn = static_cast<double>(n);
    const double inv_fn = 1.0 / fn;
    const double tol = thresh;
    const int nlam = lam_path.size();

    // Workload-Aware flag for activating Cauchy-Schwarz KKT Pruning
    const bool use_cs = (p >= 500 && n >= 150);

    // Initial capacity for active set structures - optimized to completely avoid reallocations on almost all problems
    int current_capacity = ((std::max(128, std::min(512, p)) + 7) / 8) * 8;

    // Declare raw pointers for 64-byte aligned structures
    double* G_data = nullptr;
    double* c_data = nullptr;
    double* beta_active_data = nullptr;
    double* xv_active_data = nullptr;
    double* inv_xv_active_data = nullptr;
    double* grad_init_active_data = nullptr;
    double* beta_old_at_start = nullptr;

    double* y_padded = nullptr;
    double* r_padded = nullptr;
    double* r_ref_padded = nullptr;

    bool oom = false;

    // Allocate 64-byte aligned arrays
    if (posix_memalign((void**)&G_data, 64, static_cast<size_t>(current_capacity) * current_capacity * sizeof(double))
        != 0) goto cleanup;
    if (posix_memalign((void**)&c_data, 64, static_cast<size_t>(current_capacity) * sizeof(double)) != 0) goto cleanup;
    if (posix_memalign((void**)&beta_active_data, 64, static_cast<size_t>(current_capacity) * sizeof(double)) != 0)
        goto cleanup;
    if (posix_memalign((void**)&xv_active_data, 64, static_cast<size_t>(current_capacity) * sizeof(double)) != 0) goto
        cleanup;
    if (posix_memalign((void**)&inv_xv_active_data, 64, static_cast<size_t>(current_capacity) * sizeof(double)) != 0)
        goto cleanup;
    if (posix_memalign((void**)&grad_init_active_data, 64, static_cast<size_t>(current_capacity) * sizeof(double)) !=
        0) goto cleanup;
    if (posix_memalign((void**)&beta_old_at_start, 64, static_cast<size_t>(current_capacity) * sizeof(double)) != 0)
        goto cleanup;

    if (posix_memalign((void**)&y_padded, 64, static_cast<size_t>(n_padded) * sizeof(double)) != 0) goto cleanup;
    if (posix_memalign((void**)&r_padded, 64, static_cast<size_t>(n_padded) * sizeof(double)) != 0) goto cleanup;
    if (posix_memalign((void**)&r_ref_padded, 64, static_cast<size_t>(n_padded) * sizeof(double)) != 0) goto cleanup;

```

```
    std::fill(G_data, G_data + static_cast<size_t>(current_capacity) * current_capacity, 0.0);
    std::fill(c_data, c_data + current_capacity, 0.0);
    std::fill(beta_active_data, beta_active_data + current_capacity, 0.0);
    std::fill(xv_active_data, xv_active_data + current_capacity, 0.0);
    std::fill(inv_xv_active_data, inv_xv_active_data + current_capacity, 0.0);
    std::fill(grad_init_active_data, grad_init_active_data + current_capacity, 0.0);
    std::fill(beta_old_at_start, beta_old_at_start + current_capacity, 0.0);

    std::memcpy(y_padded, y.data(), n * sizeof(double));
    for (int i = n; i < n_padded; ++i) y_padded[i] = 0.0;

    std::memcpy(r_padded, y_padded, n_padded * sizeof(double));

    // Consistently initialize r_ref_padded to y_padded (instead of all zeros) to guarantee 100% tight bounds at start
    std::memcpy(r_ref_padded, y_padded, n_padded * sizeof(double));

    // Run the solver in a nested block to make goto compile-safe
    {
        VectorXd beta = VectorXd::Zero(p);

        std::vector<char> screened(p, 0);     // 1 if screened, 0 otherwise
        std::vector<int>  active;             // indices of active features (beta != 0)
        std::vector<int>  feat_to_idx(p, -1); // maps feature to index in active set

        // Disjoint tracking partition vectors
        std::vector<int> unscreened_list(p);
        std::vector<int> screened_list(p);
        std::vector<int> screened_to_idx(p, -1);

        int unscreened_size = p;
        int screened_size = 0;
        for (int j = 0; j < p; ++j) {
            unscreened_list[j] = j;
        }

        VectorXd grad = grad_init; // grad can be modified/overwritten

        // Reference state for Cauchy-Schwarz KKT pruning
        VectorXd grad_ref;
        std::vector<double> s;
        int lambdas_since_reset = 0;

        if (use_cs) {
            grad_ref = grad_init;
            s.resize(p);
            for (int j = 0; j < p; ++j) {
                s[j] = std::sqrt(xv(j) * inv_fn);
            }
        }

        auto add_active = [&](int j) {
            if (feat_to_idx[j] != -1) return;

            // O(1) swap-deletion from screened_list to maintain partition disjointness
```

```
        int idx_in_screened = screened_to_idx[j];
        if (idx_in_screened >= 0) {
            int last_j = screened_list[screened_size - 1];
            screened_list[idx_in_screened] = last_j;
            screened_to_idx[last_j] = idx_in_screened;
            --screened_size;
            screened_to_idx[j] = -1;
        }

        int old_k = static_cast<int>(active.size());
        feat_to_idx[j] = old_k;
        active.push_back(j);
        int new_k = old_k + 1;

        if (new_k > current_capacity) {
            int new_capacity = current_capacity * 2;

            double* G_data2 = nullptr;
            double* c_data2 = nullptr;
            double* beta_active_data2 = nullptr;
            double* xv_active_data2 = nullptr;
            double* inv_xv_active_data2 = nullptr;
            double* grad_init_active_data2 = nullptr;
            double* beta_old_at_start2 = nullptr;

            if (posix_memalign((void**)&G_data2, 64, static_cast<size_t>(new_capacity) * new_capacity * sizeof(double)) != 0) { oom = true; return; }
            if (posix_memalign((void**)&c_data2, 64, static_cast<size_t>(new_capacity) * sizeof(double)) != 0) { free(G_data2); oom = true; return; }
            if (posix_memalign((void**)&beta_active_data2, 64, static_cast<size_t>(new_capacity) * sizeof(double)) != 0) { free(G_data2); free(c_data2); oom = true; return; }
            if (posix_memalign((void**)&xv_active_data2, 64, static_cast<size_t>(new_capacity) * sizeof(double)) != 0) { free(G_data2); free(c_data2); free(beta_active_data2); oom = true; return; }
            if (posix_memalign((void**)&inv_xv_active_data2, 64, static_cast<size_t>(new_capacity) * sizeof(double)) != 0) { free(G_data2); free(c_data2); free(beta_active_data2); free(xv_active_data2); oom = true; return; }
            if (posix_memalign((void**)&grad_init_active_data2, 64, static_cast<size_t>(new_capacity) * sizeof(double)) != 0) { free(G_data2); free(c_data2); free(beta_active_data2); free(xv_active_data2); free(inv_xv_active_data2); oom = true; return; }
            if (posix_memalign((void**)&beta_old_at_start2, 64, static_cast<size_t>(new_capacity) * sizeof(double)) != 0) { free(G_data2); free(c_data2); free(beta_active_data2); free(xv_active_data2); free(inv_xv_active_data2); free(grad_init_active_data2); oom = true; return; }

            std::fill(G_data2, G_data2 + static_cast<size_t>(new_capacity) * new_capacity, 0.0);

            if (old_k > 0) {
                int old_k_padded = (old_k + 7) & ~7;
                for (int col = 0; col < old_k; ++col) {
                    double* dest_col = G_data2 + col * new_capacity;
                    const double* src_col = G_data + col * current_capacity;
                    #pragma omp simd aligned(dest_col, src_col: 64)
                    for (int row = 0; row < old_k_padded; ++row) {
                        dest_col[row] = src_col[row];
                    }
                }
```

```

            #pragma omp simd aligned(c_data2, c_data: 64)
            for (int i = 0; i < old_k_padded; ++i) c_data2[i] = c_data[i];

            #pragma omp simd aligned(beta_active_data2, beta_active_data: 64)
            for (int i = 0; i < old_k_padded; ++i) beta_active_data2[i] = beta_active_data[i];

            #pragma omp simd aligned(xv_active_data2, xv_active_data: 64)
            for (int i = 0; i < old_k_padded; ++i) xv_active_data2[i] = xv_active_data[i];

            #pragma omp simd aligned(inv_xv_active_data2, inv_xv_active_data: 64)
            for (int i = 0; i < old_k_padded; ++i) inv_xv_active_data2[i] = inv_xv_active_data[i];

            #pragma omp simd aligned(grad_init_active_data2, grad_init_active_data: 64)
            for (int i = 0; i < old_k_padded; ++i) grad_init_active_data2[i] = grad_init_active_data[i];

            #pragma omp simd aligned(beta_old_at_start2, beta_old_at_start: 64)
            for (int i = 0; i < old_k_padded; ++i) beta_old_at_start2[i] = beta_old_at_start[i];
        }

        free(G_data);
        free(c_data);
        free(beta_active_data);
        free(xv_active_data);
        free(inv_xv_active_data);
        free(grad_init_active_data);
        free(beta_old_at_start);

        G_data = G_data2;
        c_data = c_data2;
        beta_active_data = beta_active_data2;
        xv_active_data = xv_active_data2;
        inv_xv_active_data = inv_xv_active_data2;
        grad_init_active_data = grad_init_active_data2;
        beta_old_at_start = beta_old_at_start2;
        current_capacity = new_capacity;
    }

    // SIMD 4x Register-Blocked Lazy Gram Precomputation (reduces column loads by 75%)
    const double* RESTRICT col_j = ASSUME_ALIGNED(X_padded + j * n_padded, 64);
    const bool run_parallel_lazy = (old_k >= 64 && static_cast<size_t>(n_padded) * old_k >= 150000);

    #pragma omp parallel for schedule(static) if(run_parallel_lazy)
    for (int i = 0; i < (old_k / 4) * 4; i += 4) {
        const double* RESTRICT col0 = ASSUME_ALIGNED(X_padded + active[i] * n_padded, 64);
        const double* RESTRICT col1 = ASSUME_ALIGNED(X_padded + active[i+1] * n_padded, 64);
        const double* RESTRICT col2 = ASSUME_ALIGNED(X_padded + active[i+2] * n_padded, 64);
        const double* RESTRICT col3 = ASSUME_ALIGNED(X_padded + active[i+3] * n_padded, 64);

        double sum0 = 0.0, sum1 = 0.0, sum2 = 0.0, sum3 = 0.0;
        #pragma omp simd reduction(+:sum0, sum1, sum2, sum3) aligned(col_j, col0, col1, col2, col3: 64)
        for (int k = 0; k < n_padded; ++k) {
            double vj = col_j[k];
            sum0 += vj * col0[k];
```

```
                sum1 += vj * col1[k];
                sum2 += vj * col2[k];
                sum3 += vj * col3[k];
            }

            double r0 = sum0 * inv_fn;
            double r1 = sum1 * inv_fn;
            double r2 = sum2 * inv_fn;
            double r3 = sum3 * inv_fn;

            G_data[old_k * current_capacity + i] = r0;
            G_data[i * current_capacity + old_k] = r0;

            G_data[old_k * current_capacity + i + 1] = r1;
            G_data[(i + 1) * current_capacity + old_k] = r1;

            G_data[old_k * current_capacity + i + 2] = r2;
            G_data[(i + 2) * current_capacity + old_k] = r2;

            G_data[old_k * current_capacity + i + 3] = r3;
            G_data[(i + 3) * current_capacity + old_k] = r3;
        }

        for (int i = (old_k / 4) * 4; i < old_k; ++i) {
            const double* RESTRICT col_act = ASSUME_ALIGNED(X_padded + active[i] * n_padded, 64);
            double dot_val = 0.0;
            #pragma omp simd reduction(+:dot_val) aligned(col_j, col_act: 64)
            for (int k = 0; k < n_padded; ++k) {
                dot_val += col_j[k] * col_act[k];
            }
            dot_val *= inv_fn;
            G_data[old_k * current_capacity + i] = dot_val;
            G_data[i * current_capacity + old_k] = dot_val;
        }
        G_data[old_k * current_capacity + old_k] = xv(j); // xv(j) is already scaled by inv_fn

        // Zero-O(n) initial correlation computation
        double sum_val = 0.0;
        const double* RESTRICT G_col = ASSUME_ALIGNED(G_data + old_k * current_capacity, 64);
        const double* RESTRICT beta_act = ASSUME_ALIGNED(beta_active_data, 64);
        #pragma omp simd reduction(+:sum_val) aligned(G_col, beta_act: 64)
        for (int i = 0; i < old_k; ++i) {
            sum_val += G_col[i] * beta_act[i];
        }
        c_data[old_k] = grad_init(j) - sum_val;

        xv_active_data[old_k] = xv(j);
        inv_xv_active_data[old_k] = 1.0 / xv(j);
        grad_init_active_data[old_k] = grad_init[j];
        beta_active_data[old_k] = 0.0;
    };

    double prev_lam = 0.0;

```

```
        // Preallocate vectors to avoid repeated heap allocation
        std::vector<int> to_activate;
        std::vector<int> screened_violators;
        std::vector<int> unscreened_violators;
        std::vector<int> to_compute;

        to_activate.reserve(p);
        screened_violators.reserve(p);
        unscreened_violators.reserve(p);
        if (use_cs) {
            to_compute.reserve(p);
        }

        for (int li = 0; li < nlam; ++li) {
            const double lam  = lam_path(li);
            const double tlam = 2.0 * lam - prev_lam;

            // ---- Step 1: Strong-rule screening (with O(1) swap-deletion) ----
            double* RESTRICT grad_ptr = grad.data();
            for (int i = 0; i < unscreened_size; ) {
                int j = unscreened_list[i];
                if (std::abs(grad_ptr[j]) > tlam) {
                    screened[j] = 1;
                    screened_to_idx[j] = screened_size;
                    screened_list[screened_size++] = j;
                    unscreened_list[i] = unscreened_list[--unscreened_size];
                } else {
                    ++i;
                }
            }

            // ---- Step 2: Outer loop ----
            int nlp = 0;
            while (true) {
                // 2a. Identify violating features among screened features
                to_activate.clear();
                const double KKT_bound_screen = lam * (1.0 + 1e-9);
                for (int i = 0; i < screened_size; ++i) {
                    int j = screened_list[i];
                    // At this point, screened_list only contains non-active screened features.
                    // Absolutely no feat_to_idx branches needed!
                    if (std::abs(grad_ptr[j]) > KKT_bound_screen) {
                        to_activate.push_back(j);
                    }
                }

                // If some screened features violate KKT, add them to active set
                if (!to_activate.empty()) {
                    for (int j : to_activate) {
                        add_active(j);
                        if (oom) goto cleanup;
                    }
                }

```

```
359             // 2b. CD over active set until convergence
360             int active_size = static_cast<int>(active.size());
361 
362             // Save beta at the start of the outer iteration to track changes
363             if (active_size > 0) {
364                 int active_size_padded = (active_size + 7) & ~7;
365                 #pragma omp simd aligned(beta_old_at_start, beta_active_data: 64)
366                 for (int i = 0; i < active_size_padded; ++i) {
367                     beta_old_at_start[i] = beta_active_data[i];
368                 }
369             }
370 
371             if (active_size > 0) {
372                 double dmax = tol; // Ensure at least one sweep
373                 while (dmax >= tol && nlp < maxit) {
374                     ++nlp;
375                     dmax = 0.0;
376                     for (int idx = 0; idx < active_size; ++idx) {
377                         const double bj_old = beta_active_data[idx];
378                         // Division-free gradient calculation
379                         const double gj     = c_data[idx] + bj_old * xv_active_data[idx];
380                         const double bj_new = soft_thresh(gj, lam) * inv_xv_active_data[idx];
381                         if (bj_new == bj_old) continue;
382                         const double delta = bj_new - bj_old;
383                         beta_active_data[idx] = bj_new;
384 
385                         // Extremely fast SIMD cache update (padded up to a multiple of 8)
386                         int active_size_padded = (active_size + 7) & ~7;
387                         double* RESTRICT c_ptr = ASSUME_ALIGNED(c_data, 64);
388                         const double* RESTRICT G_col_ptr = ASSUME_ALIGNED(G_data + idx * current_capacity,
    64);
389                         #pragma omp simd aligned(c_ptr, G_col_ptr: 64)
390                         for (int i = 0; i < active_size_padded; ++i) {
391                             c_ptr[i] -= delta * G_col_ptr[i];
392                         }
393 
394                         const double ch = xv_active_data[idx] * delta * delta;
395                         if (ch > dmax) dmax = ch;
396                     }
397                 }
398             }
399 
400             // Safety limit check
401             if (nlp >= maxit) break;
402 
403             // Incremental O(n) residual update & any_changed check (Raw-Pointer hand-vectorized loop)
404             bool any_changed = false;
405             if (active_size > 0) {
406                 double* RESTRICT r_ptr = ASSUME_ALIGNED(r_padded, 64);
407                 for (int idx = 0; idx < active_size; ++idx) {
408                     const double delta = beta_active_data[idx] - beta_old_at_start[idx];
409                     if (delta != 0.0) {
410                         const double* RESTRICT col_ptr = ASSUME_ALIGNED(X_padded + active[idx] * n_padded,
    64);
```

```
                    #pragma omp simd aligned(r_ptr, col_ptr: 64)
                    for (int i = 0; i < n_padded; ++i) {
                        r_ptr[i] -= delta * col_ptr[i];
                    }
                    any_changed = true;
                }
            }
        }

        // O(k^2) exact re-sync of correlation cache c (Sparse-Skipping Custom Loop)
        if (any_changed && active_size > 0) {
            int active_size_padded = (active_size + 7) & ~7;
            #pragma omp simd aligned(c_data, grad_init_active_data: 64)
            for (int i = 0; i < active_size_padded; ++i) {
                c_data[i] = grad_init_active_data[i];
            }
            for (int j = 0; j < active_size; ++j) {
                const double bj = beta_active_data[j];
                if (bj != 0.0) {
                    const double* RESTRICT G_col = ASSUME_ALIGNED(G_data + j * current_capacity, 64);
                    double* RESTRICT c_ptr = ASSUME_ALIGNED(c_data, 64);
                    #pragma omp simd aligned(c_ptr, G_col: 64)
                    for (int i = 0; i < active_size_padded; ++i) {
                        c_ptr[i] -= bj * G_col[i];
                    }
                }
            }
        }

        // 2c. Robust Two-Stage KKT check
        bool screened_kkt_ok = true;
        screened_violators.clear();
        const double KKT_bound = lam * (1.0 + 1e-9);

        // SIMD 4x Register-Blocked Screened KKT Checks (reduces residual vector loads by 75%)
        const double* RESTRICT r_ptr = ASSUME_ALIGNED(r_padded, 64);
        const bool run_parallel_screened = (static_cast<size_t>(n_padded) * screened_size >= 150000);

        #pragma omp parallel for schedule(static) if(run_parallel_screened)
        for (int i = 0; i < (screened_size / 4) * 4; i += 4) {
            int j0 = screened_list[i];
            int j1 = screened_list[i+1];
            int j2 = screened_list[i+2];
            int j3 = screened_list[i+3];

            const double* RESTRICT col0 = ASSUME_ALIGNED(X_padded + j0 * n_padded, 64);
            const double* RESTRICT col1 = ASSUME_ALIGNED(X_padded + j1 * n_padded, 64);
            const double* RESTRICT col2 = ASSUME_ALIGNED(X_padded + j2 * n_padded, 64);
            const double* RESTRICT col3 = ASSUME_ALIGNED(X_padded + j3 * n_padded, 64);

            double sum0 = 0.0, sum1 = 0.0, sum2 = 0.0, sum3 = 0.0;
            #pragma omp simd reduction(+:sum0, sum1, sum2, sum3) aligned(r_ptr, col0, col1, col2, col3: 64)
            for (int k = 0; k < n_padded; ++k) {
                double rk = r_ptr[k];
```

```
                sum0 += rk * col0[k];
                sum1 += rk * col1[k];
                sum2 += rk * col2[k];
                sum3 += rk * col3[k];
            }
            grad_ptr[j0] = sum0 * inv_fn;
            grad_ptr[j1] = sum1 * inv_fn;
            grad_ptr[j2] = sum2 * inv_fn;
            grad_ptr[j3] = sum3 * inv_fn;
        }

        for (int i = (screened_size / 4) * 4; i < screened_size; ++i) {
            int j = screened_list[i];
            const double* RESTRICT col_ptr = ASSUME_ALIGNED(X_padded + j * n_padded, 64);
            double dot_val = 0.0;
            #pragma omp simd reduction(+:dot_val) aligned(r_ptr, col_ptr: 64)
            for (int k = 0; k < n_padded; ++k) {
                dot_val += col_ptr[k] * r_ptr[k];
            }
            grad_ptr[j] = dot_val * inv_fn;
        }

        for (int i = 0; i < screened_size; ++i) {
            int j = screened_list[i];
            if (std::abs(grad_ptr[j]) > KKT_bound) {
                screened_violators.push_back(j);
                screened_kkt_ok = false;
            }
        }

        if (!screened_kkt_ok) {
            // Add screened violators to active set and run CD again
            for (int j : screened_violators) {
                add_active(j);
                if (oom) goto cleanup;
            }
            continue; // Skip full KKT check, go back to CD
        }

        // Only perform full KKT check on unscreened features if screened is 100% OK
        bool full_kkt_ok = true;
        unscreened_violators.clear();

        if (use_cs) {
            // Dual-Phase Adaptive Cauchy-Schwarz KKT Pruning!
            double d2 = 0.0;
            const double* RESTRICT r_curr_ptr = ASSUME_ALIGNED(r_padded, 64);
            const double* RESTRICT r_ref_ptr = ASSUME_ALIGNED(r_ref_padded, 64);
            #pragma omp simd reduction(+:d2) aligned(r_curr_ptr, r_ref_ptr: 64)
            for (int k = 0; k < n_padded; ++k) {
                double diff = r_curr_ptr[k] - r_ref_ptr[k];
                d2 += diff * diff;
            }
            double d = std::sqrt(d2);
```

```

                const double* RESTRICT grad_ref_ptr = grad_ref.data();
                const double* RESTRICT s_ptr = s.data();
                const int* RESTRICT unscreened_ptr = unscreened_list.data();

                to_compute.clear();
                for (int i = 0; i < unscreened_size; ++i) {
                    int j = unscreened_ptr[i];
                    double bound = std::abs(grad_ref_ptr[j]) + s_ptr[j] * d;
                    if (bound > KKT_bound) {
                        to_compute.push_back(j);
                    }
                }

                int num_to_compute = to_compute.size();
                bool did_reset = false;

                if (num_to_compute > 0.3 * p || lambdas_since_reset >= 8) {
                    // Drift is too large or reset interval reached, do a full reset (SIMD 4x Register-Blocked)
                    const bool run_parallel_reset = (static_cast<size_t>(n_padded) * unscreened_size >= 150000);
                    #pragma omp parallel for schedule(static) if(run_parallel_reset)
                    for (int i = 0; i < (unscreened_size / 4) * 4; i += 4) {
                        int j0 = unscreened_list[i];
                        int j1 = unscreened_list[i+1];
                        int j2 = unscreened_list[i+2];
                        int j3 = unscreened_list[i+3];

                        const double* RESTRICT col0 = ASSUME_ALIGNED(X_padded + j0 * n_padded, 64);
                        const double* RESTRICT col1 = ASSUME_ALIGNED(X_padded + j1 * n_padded, 64);
                        const double* RESTRICT col2 = ASSUME_ALIGNED(X_padded + j2 * n_padded, 64);
                        const double* RESTRICT col3 = ASSUME_ALIGNED(X_padded + j3 * n_padded, 64);
                        const double* RESTRICT r_ptr_exact = ASSUME_ALIGNED(r_padded, 64);

                        double sum0 = 0.0, sum1 = 0.0, sum2 = 0.0, sum3 = 0.0;
                        #pragma omp simd reduction(+:sum0, sum1, sum2, sum3) aligned(r_ptr_exact, col0, col1, col2, col3
    : 64)
                        for (int k = 0; k < n_padded; ++k) {
                            double rk = r_ptr_exact[k];
                            sum0 += rk * col0[k];
                            sum1 += rk * col1[k];
                            sum2 += rk * col2[k];
                            sum3 += rk * col3[k];
                        }
                        grad_ptr[j0] = sum0 * inv_fn;
                        grad_ptr[j1] = sum1 * inv_fn;
                        grad_ptr[j2] = sum2 * inv_fn;
                        grad_ptr[j3] = sum3 * inv_fn;
                    }

                    for (int i = (unscreened_size / 4) * 4; i < unscreened_size; ++i) {
                        int j = unscreened_list[i];
                        const double* RESTRICT col_ptr = ASSUME_ALIGNED(X_padded + j * n_padded, 64);
                        const double* RESTRICT r_ptr_exact = ASSUME_ALIGNED(r_padded, 64);
                        double sum = 0.0;
```

```
572                     #pragma omp simd reduction(+:sum) aligned(r_ptr_exact, col_ptr: 64)
573                     for (int k = 0; k < n_padded; ++k) {
574                         sum += r_ptr_exact[k] * col_ptr[k];
575                     }
576                     grad_ptr[j] = sum * inv_fn;
577                 }
578 
579                 std::memcpy(r_ref_padded, r_padded, n_padded * sizeof(double));
580 
581                 double* RESTRICT grad_ref_ptr_writable = grad_ref.data();
582                 #pragma omp parallel for schedule(static) if(unscreened_size >= 2048)
583                 for (int i = 0; i < unscreened_size; ++i) {
584                     int j = unscreened_ptr[i];
585                     grad_ref_ptr_writable[j] = grad_ptr[j];
586                 }
587                 lambdas_since_reset = 0;
588                 did_reset = true;
589             } else {
590                 // Compute exact gradients only for the tiny unpruned subset (SIMD 4x Register-Blocked)
591                 const bool run_parallel_comp = (num_to_compute >= 32 && static_cast<size_t>(n_padded) *
    num_to_compute >= 150000);
592                 #pragma omp parallel for schedule(static) if(run_parallel_comp)
593                 for (int k = 0; k < (num_to_compute / 4) * 4; k += 4) {
594                     int j0 = to_compute[k];
595                     int j1 = to_compute[k+1];
596                     int j2 = to_compute[k+2];
597                     int j3 = to_compute[k+3];
598 
599                     const double* RESTRICT col0 = ASSUME_ALIGNED(X_padded + j0 * n_padded, 64);
600                     const double* RESTRICT col1 = ASSUME_ALIGNED(X_padded + j1 * n_padded, 64);
601                     const double* RESTRICT col2 = ASSUME_ALIGNED(X_padded + j2 * n_padded, 64);
602                     const double* RESTRICT col3 = ASSUME_ALIGNED(X_padded + j3 * n_padded, 64);
603                     const double* RESTRICT r_ptr_exact = ASSUME_ALIGNED(r_padded, 64);
604 
605                     double sum0 = 0.0, sum1 = 0.0, sum2 = 0.0, sum3 = 0.0;
606                     #pragma omp simd reduction(+:sum0, sum1, sum2, sum3) aligned(r_ptr_exact, col0, col1, col2, col3
    : 64)
607                     for (int m = 0; m < n_padded; ++m) {
608                         double rk = r_ptr_exact[m];
609                         sum0 += rk * col0[m];
610                         sum1 += rk * col1[m];
611                         sum2 += rk * col2[m];
612                         sum3 += rk * col3[m];
613                     }
614                     grad_ptr[j0] = sum0 * inv_fn;
615                     grad_ptr[j1] = sum1 * inv_fn;
616                     grad_ptr[j2] = sum2 * inv_fn;
617                     grad_ptr[j3] = sum3 * inv_fn;
618                 }
619 
620                 for (int k = (num_to_compute / 4) * 4; k < num_to_compute; ++k) {
621                     int j = to_compute[k];
622                     const double* RESTRICT col_ptr = ASSUME_ALIGNED(X_padded + j * n_padded, 64);
623                     const double* RESTRICT r_ptr_exact = ASSUME_ALIGNED(r_padded, 64);
```

```
                    double sum = 0.0;
                    #pragma omp simd reduction(+:sum) aligned(r_ptr_exact, col_ptr: 64)
                    for (int m = 0; m < n_padded; ++m) {
                        sum += r_ptr_exact[m] * col_ptr[m];
                    }
                    grad_ptr[j] = sum * inv_fn;
                }
            }

            for (int i = 0; i < unscreened_size; ) {
                int j = unscreened_list[i];
                if (std::abs(grad_ptr[j]) > KKT_bound) {
                    screened[j] = 1;
                    unscreened_violators.push_back(j);
                    screened_to_idx[j] = screened_size;
                    screened_list[screened_size++] = j;
                    unscreened_list[i] = unscreened_list[--unscreened_size];
                    full_kkt_ok = false;
                } else {
                    ++i;
                }
            }

            if (full_kkt_ok) {
                if (!did_reset) {
                    lambdas_since_reset++;
                }
            }
        } else {
            // Standard, clean KKT check without CS pruning overhead on small/medium problems (SIMD 4x Register-Blocked)
            const bool run_parallel_uns_std = (static_cast<size_t>(n_padded) * unscreened_size >= 150000);
            #pragma omp parallel for schedule(static) if(run_parallel_uns_std)
            for (int i = 0; i < (unscreened_size / 4) * 4; i += 4) {
                int j0 = unscreened_list[i];
                int j1 = unscreened_list[i+1];
                int j2 = unscreened_list[i+2];
                int j3 = unscreened_list[i+3];

                const double* RESTRICT col0 = ASSUME_ALIGNED(X_padded + j0 * n_padded, 64);
                const double* RESTRICT col1 = ASSUME_ALIGNED(X_padded + j1 * n_padded, 64);
                const double* RESTRICT col2 = ASSUME_ALIGNED(X_padded + j2 * n_padded, 64);
                const double* RESTRICT col3 = ASSUME_ALIGNED(X_padded + j3 * n_padded, 64);
                const double* RESTRICT r_ptr = ASSUME_ALIGNED(r_padded, 64);

                double sum0 = 0.0, sum1 = 0.0, sum2 = 0.0, sum3 = 0.0;
                #pragma omp simd reduction(+:sum0, sum1, sum2, sum3) aligned(r_ptr, col0, col1, col2, col3: 64)
                for (int k = 0; k < n_padded; ++k) {
                    double rk = r_ptr[k];
                    sum0 += rk * col0[k];
                    sum1 += rk * col1[k];
                    sum2 += rk * col2[k];
                    sum3 += rk * col3[k];
                }
```

```
                    grad_ptr[j0] = sum0 * inv_fn;
                    grad_ptr[j1] = sum1 * inv_fn;
                    grad_ptr[j2] = sum2 * inv_fn;
                    grad_ptr[j3] = sum3 * inv_fn;
                }

                for (int i = (unscreened_size / 4) * 4; i < unscreened_size; ++i) {
                    int j = unscreened_list[i];
                    const double* RESTRICT col_ptr = ASSUME_ALIGNED(X_padded + j * n_padded, 64);
                    const double* RESTRICT r_ptr = ASSUME_ALIGNED(r_padded, 64);
                    double sum = 0.0;
                    #pragma omp simd reduction(+:sum) aligned(r_ptr, col_ptr: 64)
                    for (int k = 0; k < n_padded; ++k) {
                        sum += r_ptr[k] * col_ptr[k];
                    }
                    grad_ptr[j] = sum * inv_fn;
                }

                for (int i = 0; i < unscreened_size; ) {
                    int j = unscreened_list[i];
                    if (std::abs(grad_ptr[j]) > KKT_bound) {
                        screened[j] = 1;
                        unscreened_violators.push_back(j);
                        screened_to_idx[j] = screened_size;
                        screened_list[screened_size++] = j;
                        unscreened_list[i] = unscreened_list[--unscreened_size];
                        full_kkt_ok = false;
                    } else {
                        ++i;
                    }
                }
            }

            if (full_kkt_ok) {
                break; // Converged completely!
            }

            // Add unscreened violators to active set
            for (int j : unscreened_violators) {
                add_active(j);
                if (oom) goto cleanup;
            }
        }

        // Synchronize beta with beta_active and save coefficients
        for (size_t idx = 0; idx < active.size(); ++idx) {
            beta(active[idx]) = beta_active_data[idx];
        }
        coef_path.col(li) = beta;
        prev_lam = lam;
    }
}

cleanup:
```

```
    if (G_data) free(G_data);
    if (c_data) free(c_data);
    if (beta_active_data) free(beta_active_data);
    if (xv_active_data) free(xv_active_data);
    if (inv_xv_active_data) free(inv_xv_active_data);
    if (grad_init_active_data) free(grad_init_active_data);
    if (beta_old_at_start) free(beta_old_at_start);
    if (y_padded) free(y_padded);
    if (r_padded) free(r_padded);
    if (r_ref_padded) free(r_ref_padded);
}

int main() {
    int32_t n, p, n_lambda;
    if (fread(&n,        sizeof(int32_t), 1, stdin) != 1) return 1;
    if (fread(&p,        sizeof(int32_t), 1, stdin) != 1) return 1;
    if (fread(&n_lambda, sizeof(int32_t), 1, stdin) != 1) return 1;

    // X arrives row-major. Allocate RowMajor matrix to read the bytes directly!
    Eigen::Matrix<double, Eigen::Dynamic, Eigen::Dynamic, Eigen::RowMajor> X_row(n, p);
    if (fread(X_row.data(), sizeof(double), static_cast<size_t>(n) * p, stdin)
            != static_cast<size_t>(n) * p) return 1;

    // Pad row dimension of X to the multiple of 8 (guarantees perfect alignment for each column)
    int n_padded = ((n + 7) / 8) * 8;
    double* X_padded = nullptr;
    if (posix_memalign((void**)&X_padded, 64, static_cast<size_t>(n_padded) * p * sizeof(double)) != 0) return 1;

    VectorXd y(n);
    if (fread(y.data(), sizeof(double), n, stdin) != static_cast<size_t>(n)) return 1;

    VectorXd lam_path(n_lambda);
    if (fread(lam_path.data(), sizeof(double), n_lambda, stdin)
            != static_cast<size_t>(n_lambda)) return 1;

    MatrixXd coef_path = MatrixXd::Zero(p, n_lambda);

    VectorXd xv(p);
    VectorXd grad_init(p);

    const double* RESTRICT y_ptr = y.data();
    const double inv_fn = 1.0 / n;

    // 2D Cache-Blocked parallel Fused Transposition-Precomputation-Padding (FTPP)
    // Avoids separate allocation/std::fill overhead of X_padded and completely saves a full pass reading X!
    #pragma omp parallel
    {
        int nthreads = omp_get_num_threads();
        int tid = omp_get_thread_num();

        // Static partition of columns j to completely prevent thread false-sharing
        int j_per_thread = (p + nthreads - 1) / nthreads;
        int sj = tid * j_per_thread;
        int ej = std::min(sj + j_per_thread, p);
```

```
785
786         if (sj < ej) {
787             const int col_block = 64;
788             const int row_block = 64;
789             for (int bj = sj; bj < ej; bj += col_block) {
790                 int lim_j = std::min(bj + col_block, ej);
791
792                 double local_xx[64] = {0.0};
793                 double local_xy[64] = {0.0};
794
795                 for (int bi = 0; bi < n; bi += row_block) {
796                     int lim_i = std::min(bi + row_block, n);
797                     for (int j = bj; j < lim_j; ++j) {
798                         int local_j = j - bj;
799                         double* RESTRICT dest = X_padded + j * n_padded;
800                         const double* RESTRICT src = X_row.data() + j;
801
802                         double sum_xx = 0.0;
803                         double sum_xy = 0.0;
804                         #pragma omp simd reduction(+:sum_xx, sum_xy)
805                         for (int i = bi; i < lim_i; ++i) {
806                             double val = src[i * p];
807                             dest[i] = val;
808                             sum_xx += val * val;
809                             sum_xy += val * y_ptr[i];
810                         }
811                         local_xx[local_j] += sum_xx;
812                         local_xy[local_j] += sum_xy;
813                     }
814                 }
815
816                 // Set the padded elements of each column to 0.0, and store precomputed xv and grad_init
817                 for (int j = bj; j < lim_j; ++j) {
818                     double* RESTRICT dest = X_padded + j * n_padded;
819                     for (int i = n; i < n_padded; ++i) {
820                         dest[i] = 0.0;
821                     }
822                     xv(j) = local_xx[j - bj] * inv_fn;
823                     grad_init(j) = local_xy[j - bj] * inv_fn;
824                 }
825             }
826         }
827     }
828
829     // Immediately free memory of X_row to minimize memory footprint
830     X_row.resize(0, 0);
831
832     const double thresh = 1e-9;
833     const int    maxit  = 100000;
834
835     solve_active_set(X_padded, n_padded, n, p, y, lam_path, xv, grad_init, coef_path, thresh, maxit);
836
837     fwrite(coef_path.data(), sizeof(double),
838            static_cast<size_t>(p) * n_lambda, stdout);
```

```
839
840     free(X_padded);
841     return 0;
842 }
843 '''
844
845 COMPILE_FLAGS = ["-fopenmp", "-ffast-math"]
846
847 # EVOLVE-BLOCK-END
```